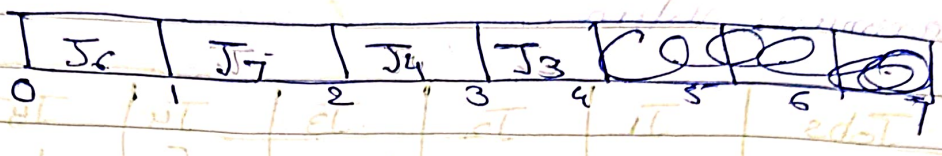
ex 2

Jobs	1	2	3	4	5	6
Profit	3	5	20	18	0	6
Deadline	1	3	4	3	2	1

Step 1 :- Arrange in descending order.

Job	7	J_3	J_4	6	2	1	5
Profit	30	20	18	6	5	3	0
Deadline	2	4	3	1	3	1	2



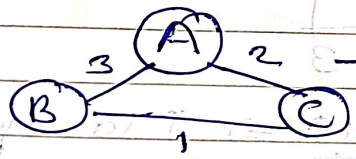
$\therefore 30 + 20 + 18 + 6 = 74 \Rightarrow$ Max profit.

Not in MISE

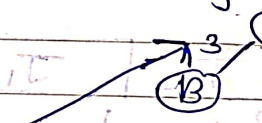
Minimum cost spanning tree : Greedy approch

Spanning tree

- A spanning tree is undirected & acyclic tree/ graph



→ this not spanning tree



→ this is spanning tree

there are 3 vertices & edge is 2

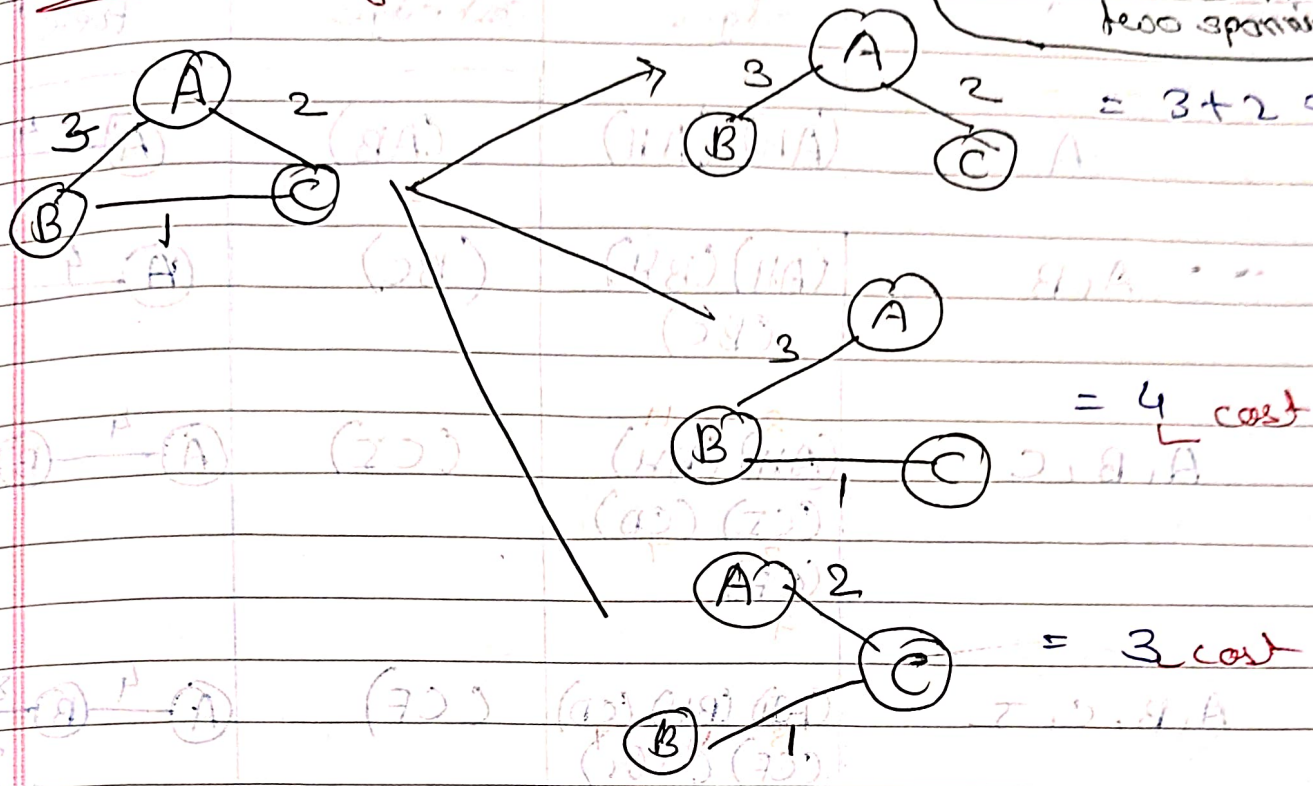
Note :- A graph without a cycle is called spanning tree

- Spanning tree is sub graph
- Spanning tree contain all vertices
- No cycle.

} What is spanning tree

Note:- edge = Vertice - 1

Note:- if cost of two tree is same then there is two spanning tree



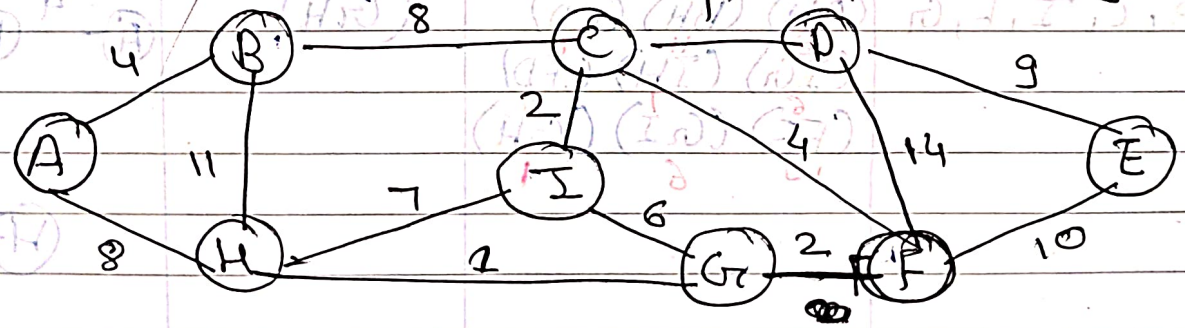
∴ 3 is the Min.m cost (optimal sol.n)

There are two algo.

① PRIMS
② Kruskal —

① PRIMS algorithm :-

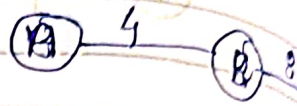
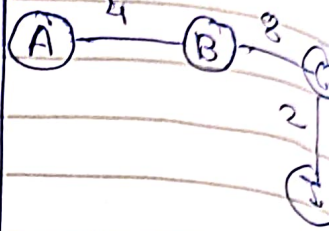
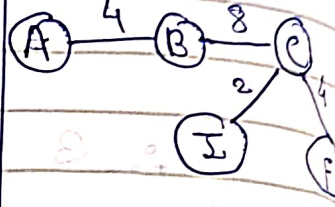
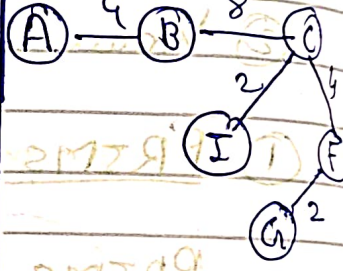
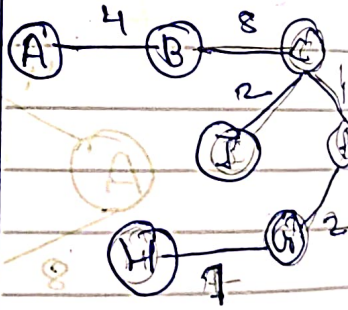
- PRIMS alg. to find min.m cost spanning tree.

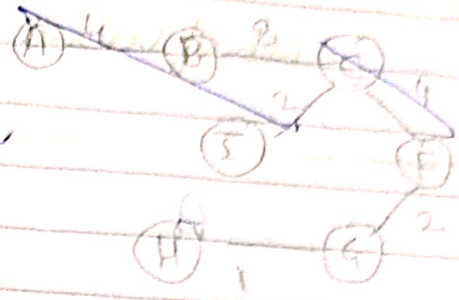
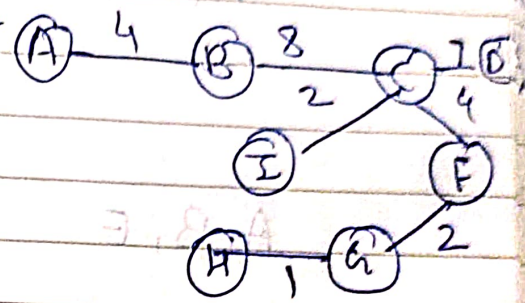
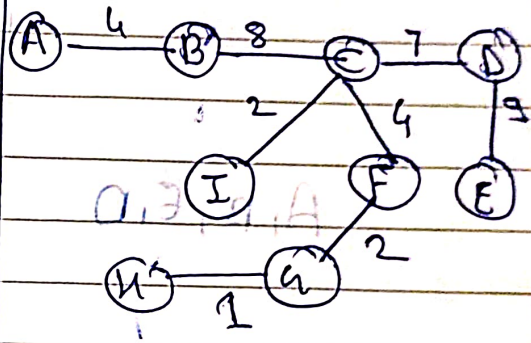


Q Find out the min.m cost spanning tree by using prims algorithm (with starting vertex)

① Select starting vertice
② Select adjustion vertices

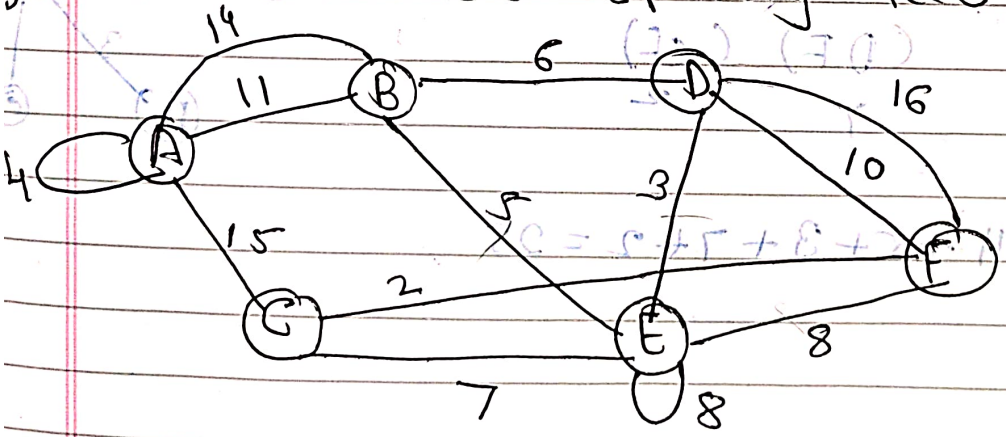
Soln :-

set of vertices	Adjustent edges	Min. cost edge	Partial spanning tree
A	(AB) (AH)	(AB)	
A, B	(AH) (BH) (BC)	(BC)	
A, B, C	(AH) (BH) (CI) (CD) (CF)	(CI)	
A, B, C, I	(AH) (BH) (CD) (CF) (IG) (IH)	(CF)	
A, B, C, I, F	(AH) (BH) (CD) (IG) (IH) (FD) (GF) (FE)	(GF)	
A, B, C, I, F, G	(AH) (BH) (CD) (IG) (IH) (FD) (FE) (GI) (GH)	(GH)	

A, B, ~~C~~, ~~I~~, F, G, H	(AH) (BH) ~~(CD)~~	(HG)	
	(IG) (IH) (FD)		
	(FE) ~~(GI)~~ (HG)		
A, B, C, I, F, G, H, E	(AH) (BH) ~~(CD)~~	(CD)	
	(IG) (IH) (FD)		
	(FE) (GI) ~~(DE)~~		
B, C, I, F, H, D	(DE) (AH) (BH)		
	(IG) (IH) (FD)		
	(GI) ~~(AB) (CE)~~		
	(FE)		

$\therefore 4+8+7+9+2+2+1+2 =$

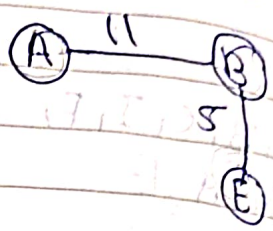
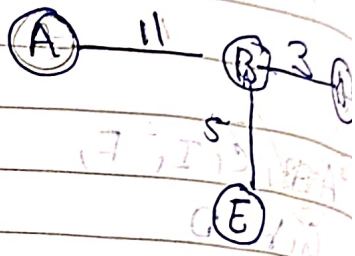
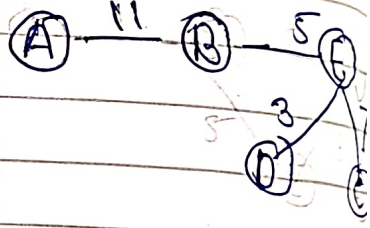
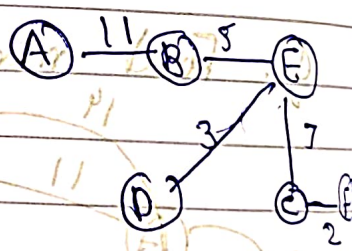
Q. Find min.m cost spanning tree.



Note: ① does not consider loop. bca it is cyclic

② if there is parallel vertice then select min.m vertice

Soln:-

Set of vertices	Adjustance edges	Min.m cost edge	Partial spanning tree
A	(AB) 11 (AC) 15	(AB)	
A, B	(AC) 15, (BD) 6, (BE) 5	(BE)	
A, B, E	(AC) 15, (BD) 6, (ED) 3, (EC) 7, (EF) 8	(ED)	
A, B, E, D	(AC) 15, (EF) 8, (EE) 7, (DF) 10	(EE)	
A, B, E, D, C	(AC) 15, (EF) 8, (DF) 7, (CF) 2	CF	

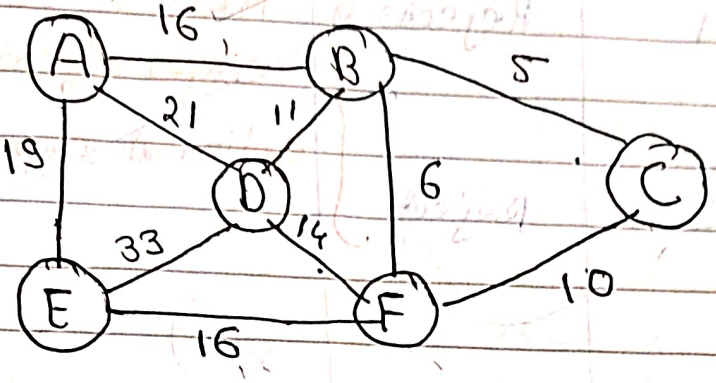
$\therefore 11 + 5 + 3 + 7 + 2 = 28$

Unit-3 :- Continued

Kruskal's algorithm

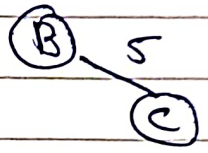
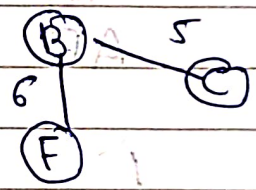
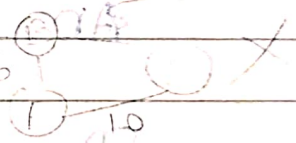
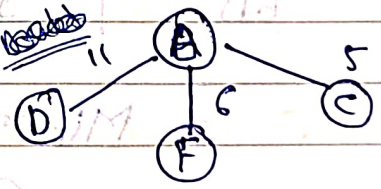
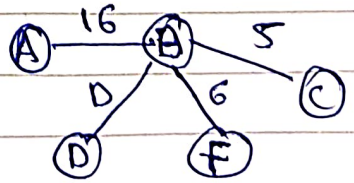
To find the minimum cost spanning tree.

ex:- (1)

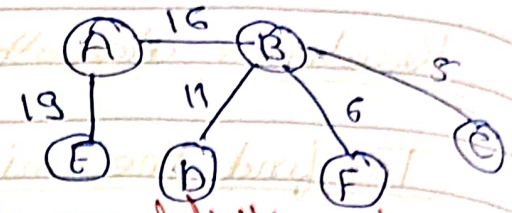


Note:- If it is not contain cycle, we will accept else reject (if it contain cycle)

Step 1:- Arrange all edges in ascending order.

	Edges	Cost	Accept/Reject	Subgraph
(Min cost vertice)	BC	5	Accept	
	BF	6	Accept	
(it form cycle, reject it)	FC	10	Reject	
	BD	11	Accept	
(cycle)	DF	14	Reject	
	AB	16	Accept	

AE	19	Accept
AD	21	Reject
ED	33	Reject

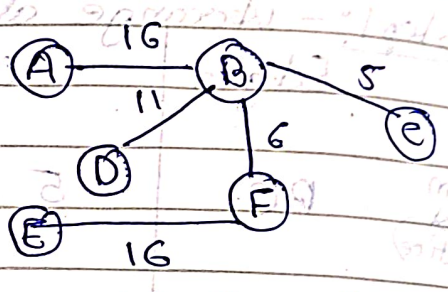


← As per definition of spanning tree it cover all the edges of min.m cost

optional step.

→

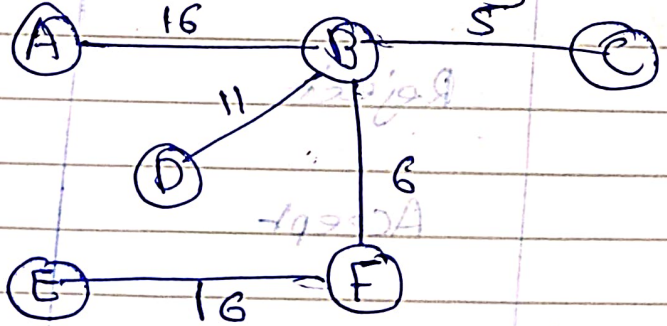
EF	16	Accept
AE	19	Reject
AD	21	Reject
ED	33	Reject



As per definition of spanning tree it cover all the edges of min.m cost.

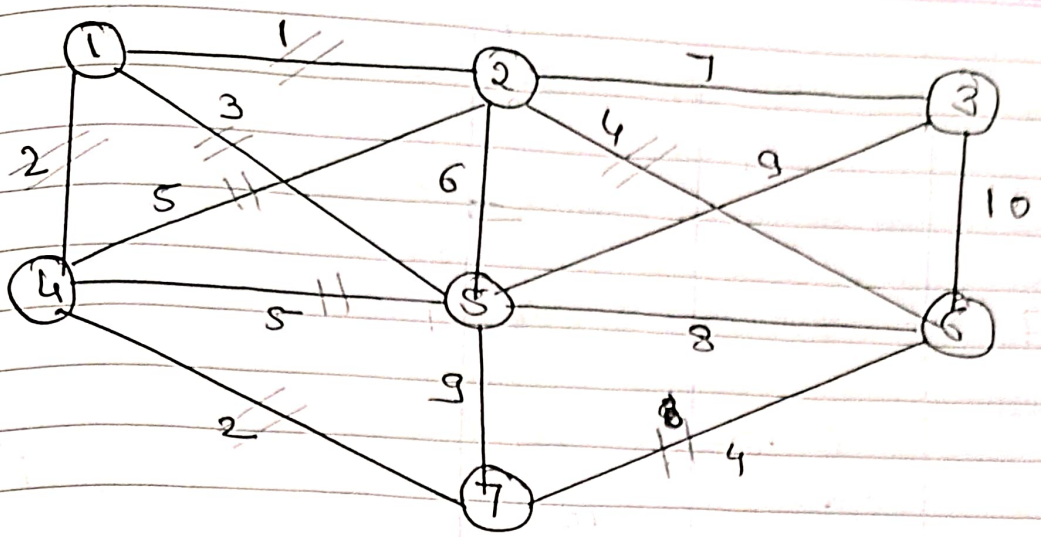
optional step

∴ Min.m cost spanning free

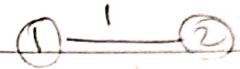
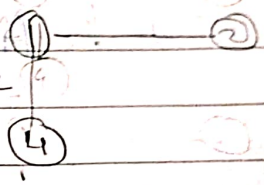
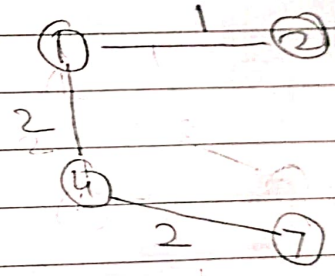
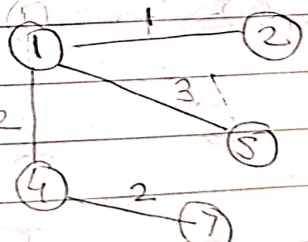
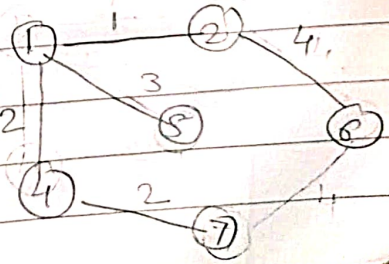


∴ Total cost is = 54

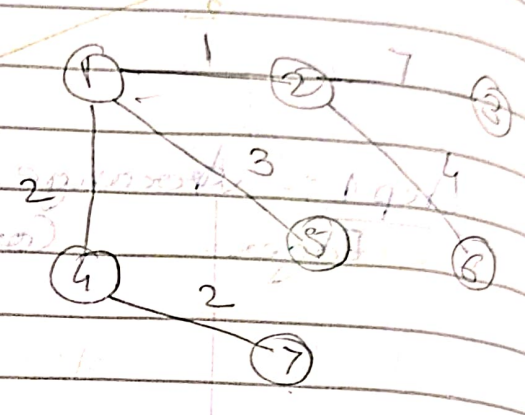
Eg (2)



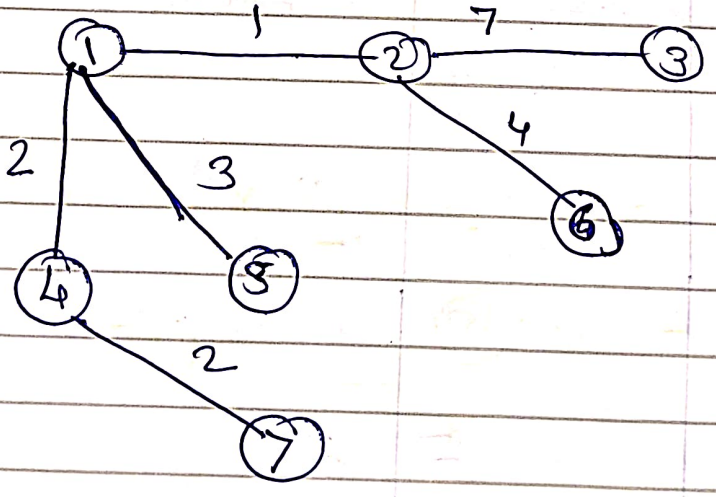
step 1 :- Arrange all edges in ascending order.

Edges	Cost	Accept / Reject	subgraph
12	1	Accept	
14	2	Accept	
47	2	Accept	
15	3	Accept	
26	4	Accept	

67	4	Reject
42	5	Reject
45	5	Reject
25	6	Reject
23	7	Accept



∴ As per definition of spanning tree, it covers all the edges of Min.m cost
∴ Min.m cost spanning tree



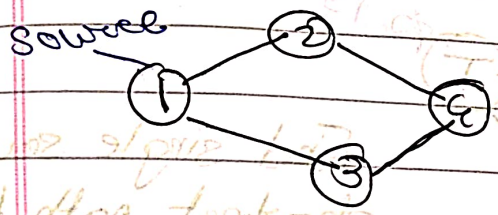
Total cost = 1+7+2+3+2+4
= 19

Greedy approch : Single source shortest Path

① **Dijikstra Algorithm** : ① It work in undirected or directed graph
② It work for positive weight +ve cost edges
③ This follow greedy approach

Path problem
- Single Source Shortest path
- All Pair shortest path

① **Single source shortest path** (min^m cost between single cost)

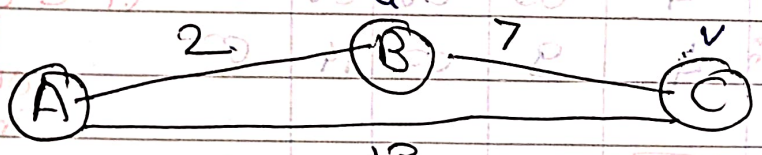


1-2 path
1-3 path
> Single source shortest path.

② **All pair shortest path** (min^m cost between all node/ source)

To calculate a find shortest path we see all pair of node.

* It work on property **Relaxation** it is applied to both graph i.e directed & undirected



Assume A-B = ~~∞~~ 2
A-C = ~~∞~~ 9 i.e A-B-C = 2+7 = 9 ✓
A-C = 13 ×

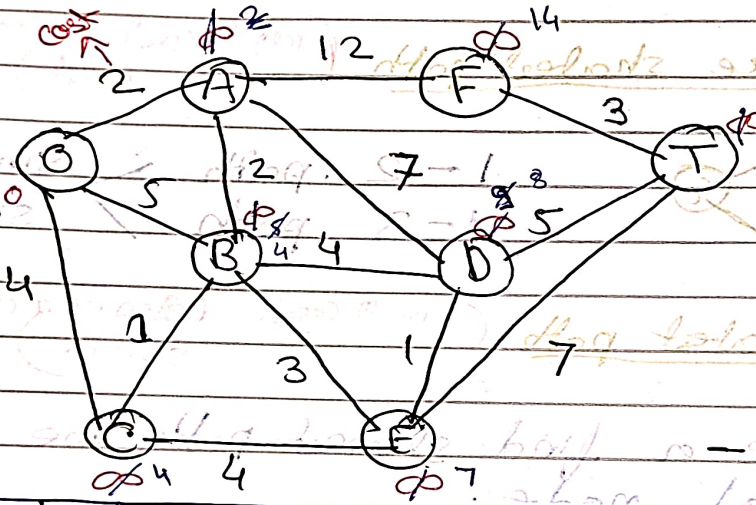
Note:- Relaxation said whoever the min.m cost choose that.

formula for relaxation

distance	cost	
d(u)	+ c(u,v)	d(v)
2	+ 7	13
9		13

where u = intermiate node
v = goal / final node
d = distance
c = cost

[Eg]



Find single source shortest path by using dijkstra algorithm.

— Source is O

write direct distance

	O	A	B	C	D	E	F	T
O	0	∞	∞	∞	∞	∞	∞	∞
A		2	5 (by relaxation 2)	4	∞	∞	∞	∞
B			2	4	9	∞	14	∞
C				4	8	7	14	∞
E					8	7	14	∞
D					8		14	13
T							14	13
F							14	

(i.e OA, OB, OC is direct, OD, OE, OF, OT is indirect, so it is ∞)

← select anyone

Note :- if there is no direct relation, make it ∞
if there is direct relation, then write the given cost

0-A-C		0-A-B
d(A) + C(A,C)	d(C)	d(A) + C(A,B)	d(B)
2 + ∞	4	2 + 2	5
2	4	(4)	5

There is no direct relation ∴ ∞

The distance betn A & D is not direct, so cost is also not direct

(0-A-D)		(0-A-E)	
d(A) + C(A,D)	d(D)	d(A) + C(A,E)	d(E)
2 + 7	∞	2 + ∞	∞
(9)	∞	(∞)	∞

For F (0-A-F)

d(A) + C(A,F)	d(F)
2 + 12	∞
14	∞

For T (0-A-T) There is no direct connection
d(A) + C(A,T) ∴ infinite

(I) 0-B = 4

(II) 0-B-C

d(B) + C(B,C)	d(C)
4 + 1	4
5	(4)

(III) 0-B-D (0-D = ∞, 0-A-D = 9)

d(B) + C(B,D)	d(D)
4	9
(8)	9

(iv) 0 - E

d(B)	+	c(B,E)	d(E)
4	+	3	∞
7			

(v) 0 - F (there is no direct relationship, so go path the vertex reaches)

d(B)	+	c(B,F)	d(F)
4	+	∞	14
∞			14

(vi) 0 - F ∴ no direct cost ∴ infinite

(A) 0 - C

0 - C - D (no direct cost put as it is)

d(C)

∞

(B) 0 - C - E

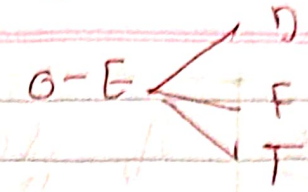
d(C)	+	c(C,E)	d(E)
4	+	4	7
8			7

(C) 0 - C - F

d(C)	+	c(C,F)	d(F)
4	+	∞	14
∞			14

(D) 0 - C - T

d(C)	+	c(C,T)	d(T)
			∞



(A) O-E-F

d(E)	+	c(E,F)	d(F)
7	+	∞	14
7			14

(B) O-E-T

d(E)	+	c(E,T)	d(T)
7	+	7	∞
14			∞

(1) (O-A-F) = 14 There is no direct relationship so put ∞ as it is = 14

C

- O-A = 2
- O-B = 4
- O-A-B = 4
- O-D = 8
- O-A-B-D = 8
- O-A-B-E = 7
- O-A-F = 14
- ~~O-T = 13~~
- ~~O-B-D-T = 13~~
- ~~O-A-B.~~
- O-A-B-D-T = 13

(M-1) Q What is relaxation in shortest path?

Date 10/5

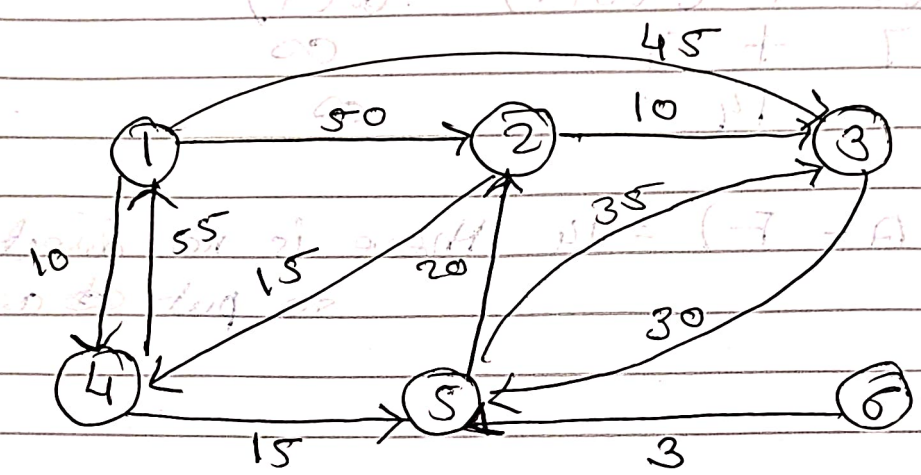
d(A) + c(A,B) d(B)
2 2 5

Ip

If $d(u) + c(u,v) < d(v)$
Then
$d(v) = d(u) + c(u,v)$
else
$d(v) = d(v)$

Relaxation

Q



Here, source is 1

Note:- there is no connection between (1 to 6) so all relation bet.ⁿ 1 to 6 is ∞

1 - 2 (1 - 4 - 5 - 2 = 45)

1 - 3

1 - 4

1 - 5

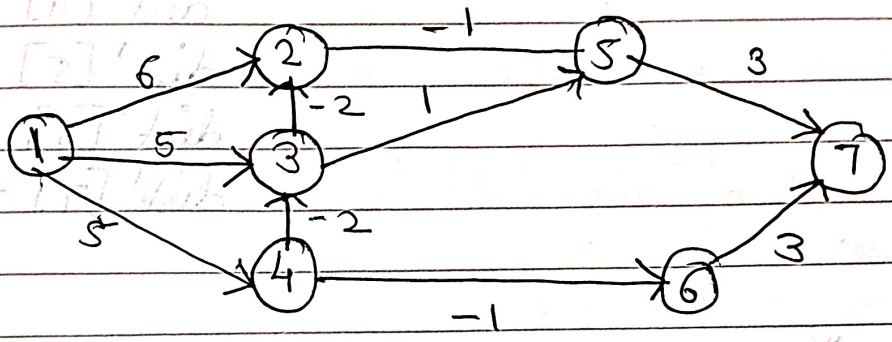
1 - 6 = ∞

(2) (Bellman ford algorithm) – Single source shortest path

It work in directed graph
It also work in negative edges / cost / weight.
This follow dynamic approach.

Q find the single source shortest path for the following graph using Bellman ford algorithm.

[M-5] [M-3] [M-4]



$$dist^{k}[u] = \min\{ dist^{k-1}[u], \min\{ dist^{k-1}[i] + cost[i,u] \}$$

Soln:-

Distance matrix
u ← dist [1 - - - - 7]

k ↓	1	2	3	4	5	6	7
1	0	6	5	5	∞	∞	∞
2	0	3	3	5	5	4	∞
3	0	1	3	5	2	4	7
4	0	1	3	5	0	4	5
5	0	1	3	5	0	4	3
6	0	1	3	5	0	4	3

← final onces

- In column, total 7 box, in row [-1] i.e 6 box
- 1 to 5 is not direct ∴ it is ∞.

1 1 1

5-2 = 3

1-4-3-

Note:- distance see from matrix & cost see from given graph.

$dist^k[u] = \min\{ dist^{k-1}[u], \min\{ dist^{k-1}[i] + cost[i,u] \}\}$

skip

Here, $u = 2$, $k = 1$ $\quad i = 1, \cancel{2}, 3, 4, 5, 6, 7$

$k-1 = 1-1 = 0$ $\qquad$ $2-1 = 1$

$dist^2[2] = \min\{ dist^1[2], \min\{ dist^1[1] + cost[1,2]$
$dist^1[3] + cost[3,2]$
$dist^1[4] + cost[4,2]$
$dist^1[5] + cost[5,2]$
$dist^1[6] + cost[6,2]$
$dist^1[7] + cost[7,2]$

$= \min\{ 6, \min\{ 0+6, 5+(-2), 5+(\infty), \infty+(\infty), \infty+(\infty), \{\infty\}+(\infty) \}\}$

$= \min\{ 6, \min\{ 6, 3, \infty, \infty, \infty \cdots \}\}$

$\min\{ 6, 3 \}$

$= \min\{3\}$

$= 3$

Here, $k = row = 2$ $\quad u = 3$ $\quad i = 1, 2, 4, 5, 6, 7$

$2-1 = 1$

$dist^2[3] = \min\{ dist^1[3], \min\{ dist^1[1] + cost$
$dist^1[2] + cost[2,3]$
$dist^1[4] + cost[4,3]$
$dist^1[5] + cost[5,3]$
$dist^1[6] + cost[6,3]$
$dist^1[7] + cost[7,3]\}\}$

$= \min\{5, \min\{0+(5), 6+\infty, 5+(-2)+\infty, \infty+\infty, \infty\}\}$

$= \min\{5, \min\{5, \infty, 3, \infty, \infty, \infty\}\}$

$= \min\{5, \min\{3\}\}$

$= \min\{5, 3\}$

$= \underline{3}$

Here $k=2$, $u=4$, $i = 1, 2, 3, 5, 6, 7$

$dist^2[4] = \min\{dist^1[4], \min\{dist^1[1] + cost(1,4), dist^1[2] + cost(2,4), dist^1[3] + cost[3,4], dist^1[5] + cost[5,4], dist^1[6] + cost[6,4], dist^1[7] + cost[7,4]\}\}$

$= \min\{5, \min\{0+5, 6+\infty, 5+\infty, \infty+\infty, \infty+\infty, \infty+\infty\}\}$

$= \min\{5, \min\{5\}\}$

$= \min\{5, 5\}$

$= \min\{5\}$

$= \underline{5}$

Here $k=2$, $u=5$, $i = 1, 2, 3, 4, 6, 7$

$dist^2[5] = \min\{dist^1[4], \min\{dist^1[1] + cost(1,5), dist^1[2] + cost(2,5), dist^1[3] + cost(3,5), dist^1[4] + cost(4,5)\}\}$

dist'[6] + cost(6,5)
dist'[7] + cost(7,5)

$= \min\{\infty^5, \min\{0+\infty, 6+\infty, 5+1, 5+\infty, \infty+\infty\}\}$

$= \min\{5, 6\}$
$= 5$

Here, $k=2$, $n=6$, $i = 1,2,3,4,5,7$

dist'[6] = min{ dist'[6] + 0, min{ dist'[1] + cost(1,6)
dist'[2] + cost(2,6)}
dist'[3] + cost(3,6)}
dist'[4] + cost(4,6)}
dist'[5] + cost(5,6)}
dist'[7] + cost(7,6)}

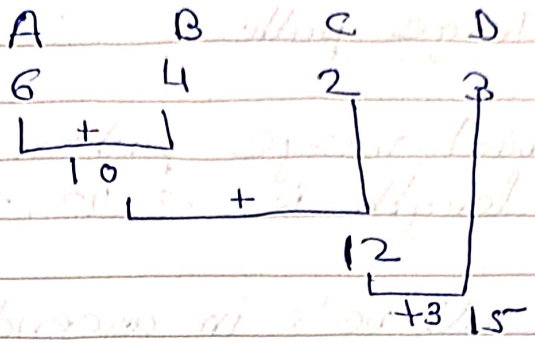
$= \min\{\infty, \min\{0+\infty, 6+\infty, 5+\infty, 5+\infty, \infty, \infty\}\}$

Greedy approach ⟸ min cost spanning tree
Job sequence
Krushal
Dijsha

1) optimal merge pattern
2) Huffman coding

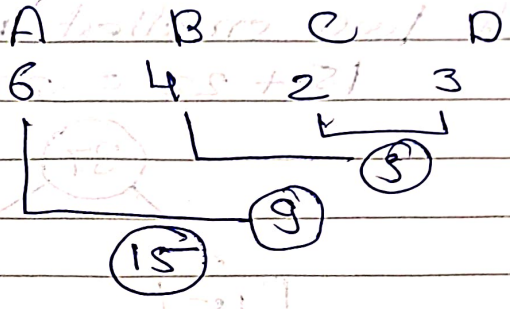
Merge array A B = C
m n = m+n

[9] Method I



10 + 12 + 15 = 37

Method II

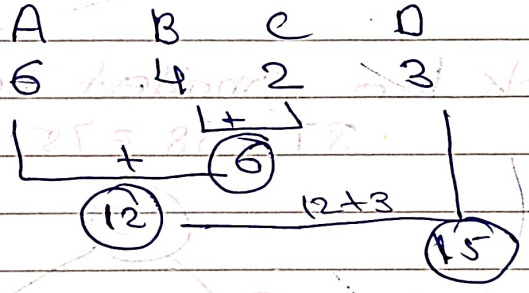


5 + 9 + 15 = 29

↓

this is a optimized way to merge sort

Method III



= 6 + 12 + 15 = 33

RULES

1. first arrange no. in ascending order
2. add two smallest element 1st
3. After adding two element, add that new number with 2nd smallest number.

① Optimal Merge pattern

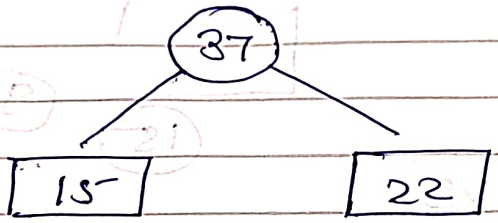
Q Find optimal merge pattern for 10 file whose length is 38, 42, 22, 15, 94, 63, 101, 43

step 1 :- Arrange all no's in ascending order

~~38~~ ~~15~~ 22 38 42 43 63 94 101

step 1.1 pick two smallest number & add those

15 + 22 = 37

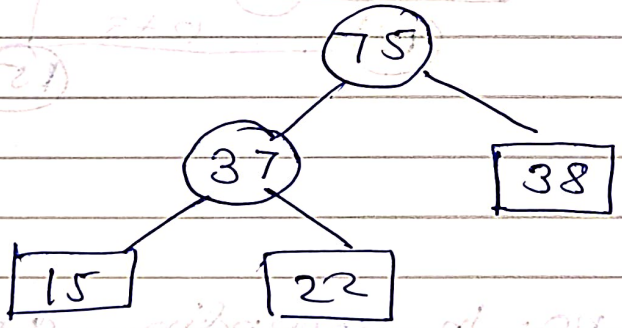


step 2 :- Arrange all no's in ascending order

~~37~~ ~~38~~ 42 43 63 94 101

step 2.1 :- pick two smallest no. & add those

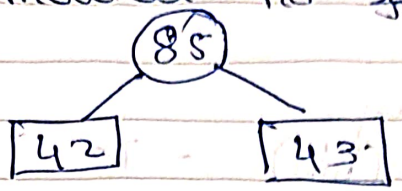
37 + 38 = 75



step 3 :- Arrange in ascending order

42 43 63 75 94 101

step 3.1 pick two smallest no & add those

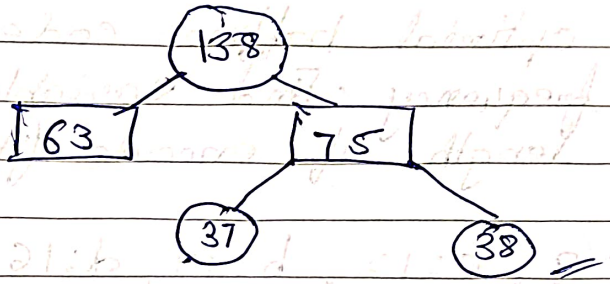


step 4:- Arrange no in ascending order

~~63~~ ~~75~~ 85 94 101

step 4.1 :- pick two smallest no's & add those

63 + 75 = 138



step 5 :- Arrange ~~85~~ ~~94~~ 101 138

step 5.1 85 + 94 = 179

Huffman code

letter	frequency	
N	10 x 7 (bits)	= 70
A	20 x 7	= 140
G	30 x 7	= 21
P	9 x 7	= 63
U	4 x 7	= 28
R	2 x 7	= 14
		336

Eg] Q. In a optimal huffman code for the follow set of frequency. Find variable length code. word & length of encode file.

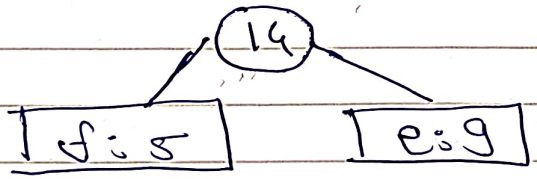
letter ↑ frequency ↑

f:5, e:9, c:12, b:13, d:16, a:45

Soln:- step1: Arrange all frequencies in ascending order.

Here frequencies is already in ascending orde

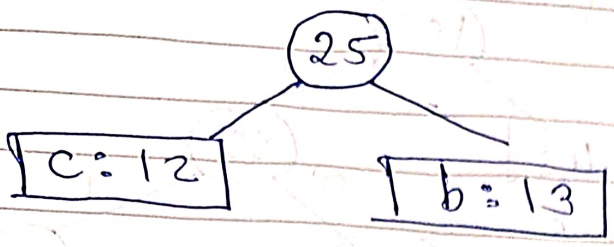
step2:- Add the two number



step2:- Arrange the number in ascending order

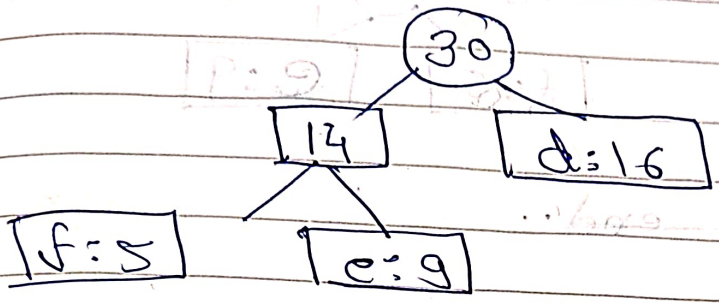
c:12, b:13, 14, d:16, a:45

Step2) Add two min.m number

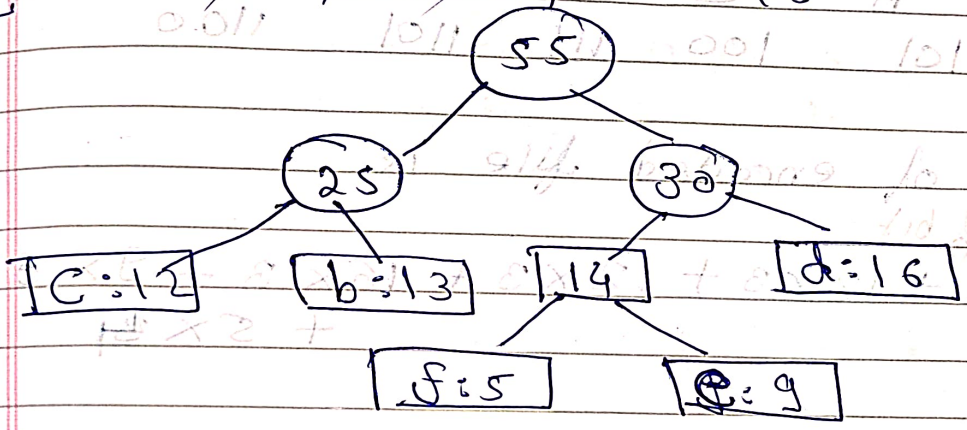


Step3:- Arrange the no in ascending order & choose min.m numbers & add them.

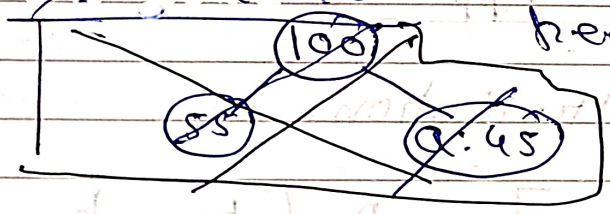
14, d:16, 25, a:45



Step4:- 25, 30, a:45

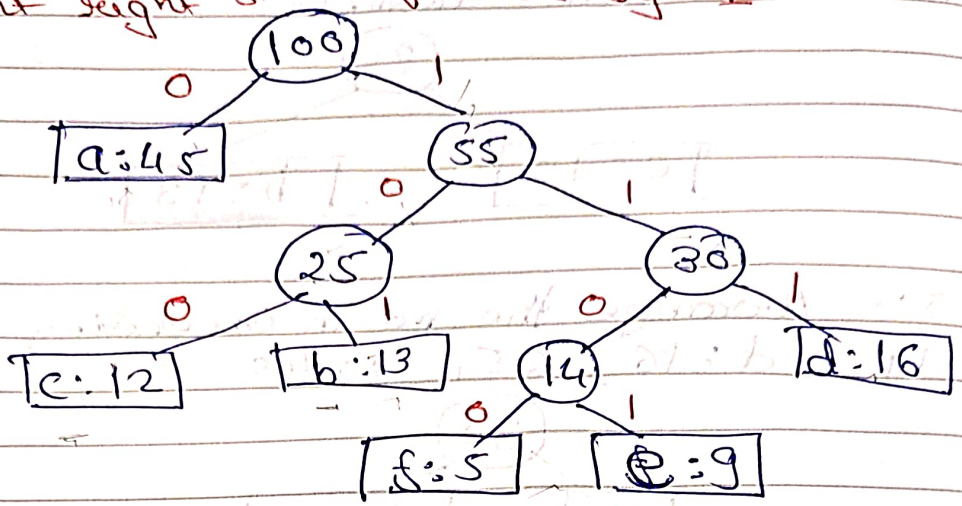


Step5:- Arrange in ascending order 55, a:45



tree see in next page

Represent left side of node by 0
Represent right side of node by 1



~~Step6: Letter letter are end..~~

Step6 :- Letter are encoded as

a	b	c	d	e	f
0	101	100	111	1101	1100

∴ length of encoded file is.
frequency bit

$45 \times 1 + 13 \times 3 + 12 \times 3 + 16 \times 3 + 9 \times 4 + 5 \times 4$

$= 224$

Divide & Conquer Approach

Problem : Matrix Multiplication.

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}_{2\times 2} \qquad B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}_{2\times 2}$$

$A \times B = C$

$$C = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

$c_{11} = a_{11} \times b_{11} + a_{12} \times b_{21}$ (1) (2)

$c_{12} = a_{11} \times b_{12} + a_{12} \times b_{22}$ (3) (4)

$c_{21} = a_{21} \times b_{11} + a_{22} \times b_{21}$ (5) (6)

$c_{22} = a_{21} \times b_{12} + a_{22} \times b_{22}$ (7) (8)

Total 8 multiplications to get 2×2 matrix.

Algorithm Matrix:-

Matrix $A = \left[\begin{array}{cc|cc} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ \hline a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{array}\right]$ (A_1, A_2, A_3, A_4)

$B = \left[\begin{array}{cc|cc} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ \hline b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{42} & b_{43} & b_{44} \end{array}\right]$ (B_1, B_2, B_3, B_4)

$$\left[\begin{array}{c|c} A_1 B_1 & A_2 B_2 \\ \hline A_3 B_3 & A_4 B_4 \end{array}\right]$$

Algorithm

```
matrix multiplication (A, B, n)
    if (n <= 2)
    {
     c11 = a11 .
     c12
     c21
     c22
    }
```

else
{
mid = n/2
mm = (A1, B1, n/2)
nm = (A2, B2, n/2)
mm = (A3, B3, n/2)
nm = (A4, B4, n/2)
}

Complexity: $T(n) = 8T\left(\frac{n}{2}\right) + n^2$

Strassen's Matrix Multiplication

⇒ this is use to reduce $8T\left(\frac{n}{2}\right)+n^2$ into $7T\left(\frac{n}{2}\right)+n^2$

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

Resultant Matrix $C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$

$P = (A_{11} + A_{22})(B_{11} + B_{22})$
$Q = B_{11}(A_{21} + A_{22})$
$R = A_{11}(B_{12} - B_{22})$
$S = A_{22}(B_{21} - B_{11})$
$T = B_{22}(A_{11} + A_{12})$
$U = (B_{11} + B_{12})(B_{21} - A_{11})$
$V = (B_{21} + B_{22})(A_{12} - A_{22})$

$C_{11} = P + S - T + V$

$C_{12} = R + T$

$C_{21} = Q + S$

$C_{22} = P + R - Q + U$

Here, we will reduce the 8 to 7

i.e $T(n) = 8T\left(\frac{n}{2}\right) + n^2$

to 7

Q Multiply matrix using strassen's matrix multiplication & find out the resultant matrix

$$A = \begin{bmatrix} 1 & 2 \\ 7 & 5 \end{bmatrix} \qquad B = \begin{bmatrix} 6 & 7 \\ 3 & 8 \end{bmatrix} \qquad C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

→ $P = (A_{11} + A_{22})(B_{11} + B_{22})$

$= (1 + 5)(6 + 8)$

$= (6) \times (14)$

$= 84$

$S = A_{22}(B_{21} - B_{11})$

$= 5(3 - 6)$

$= 5(-3)$

$= -15$

$T = B_{22}(A_{11} + A_{12})$

$= 8(1 + 2)$

$= 24$

$$N = (B_{21} + B_{22})(A_{12} - A_{22})$$
$$= (3+8)(2-5)$$
$$= (11)(-3)$$
$$= -33$$

$$Q = B_{11}(A_{21} + A_{22})$$
$$= 6(7+5)$$
$$= 6(12)$$
$$= 72$$

$$R = A_{11}(B_{12} - B_{22})$$
$$= 1(7-8)$$
$$= -1$$

$$U = (B_{11} + B_{12})(B_{21} - A_{11})$$
$$= (6+7)(3-1)$$
$$= (13)(2)$$
$$= 26$$

$$C_{11} = 84 + (-15) - 24 + (-33)$$
$$= 84 - 15 - 24 - 33$$
$$= 12$$

$$C_{12} = -17 + 24$$
$$= 23$$

$$C_{21} = 72 - 15$$
$$= 57$$

$C_{22} = P + R - Q + U$

$= 84 + (-1) - 72 + 26$

$= 84 - 1 - 72 + 26$

$=$

$C = \begin{bmatrix} 19 & 11 & 22 & 54 \\ -42 & 13 & 78 & 89 \\ 81 & 89 & 11 & -111 \\ 11 & 5 & 97 & 59 \end{bmatrix}$

Q. $A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 2 & 7 & 1 \\ 2 & 7 & 0 & 5 \\ 4 & 3 & 2 & 1 \end{bmatrix}$ $B = \begin{bmatrix} 5 & 6 & 7 & 8 \\ 1 & 0 & 3 & 4 \\ 6 & 2 & 7 & 0 \\ 8 & 1 & 6 & 2 \end{bmatrix}$

$C = \begin{bmatrix} C_{11} & C_{12} & C_{13} & C_{14} \\ C_{21} & C_{22} & C_{23} & C_{24} \\ C_{31} & C_{32} & C_{33} & C_{34} \\ C_{41} & C_{42} & C_{43} & C_{44} \end{bmatrix}$

$P = (A_{11} + A_{22})(B_{11} + B_{22})$

$= \left(\begin{bmatrix} 1 & 2 \\ 5 & 2 \end{bmatrix} + \begin{bmatrix} 0 & 5 \\ 2 & 1 \end{bmatrix} \right) \left(\begin{bmatrix} 5 & 6 \\ 1 & 0 \end{bmatrix} + \begin{bmatrix} 7 & 0 \\ 6 & 2 \end{bmatrix} \right)$

$= \begin{bmatrix} 1 & 7 \\ 7 & 8 \end{bmatrix} \begin{bmatrix} 12 & 6 \\ 7 & 2 \end{bmatrix}$

$= 61$

$Q = B_{11}(A_{21} + A_{22}$

22/9/25

Dynamic Programming

Multistage Graph

Forward approach

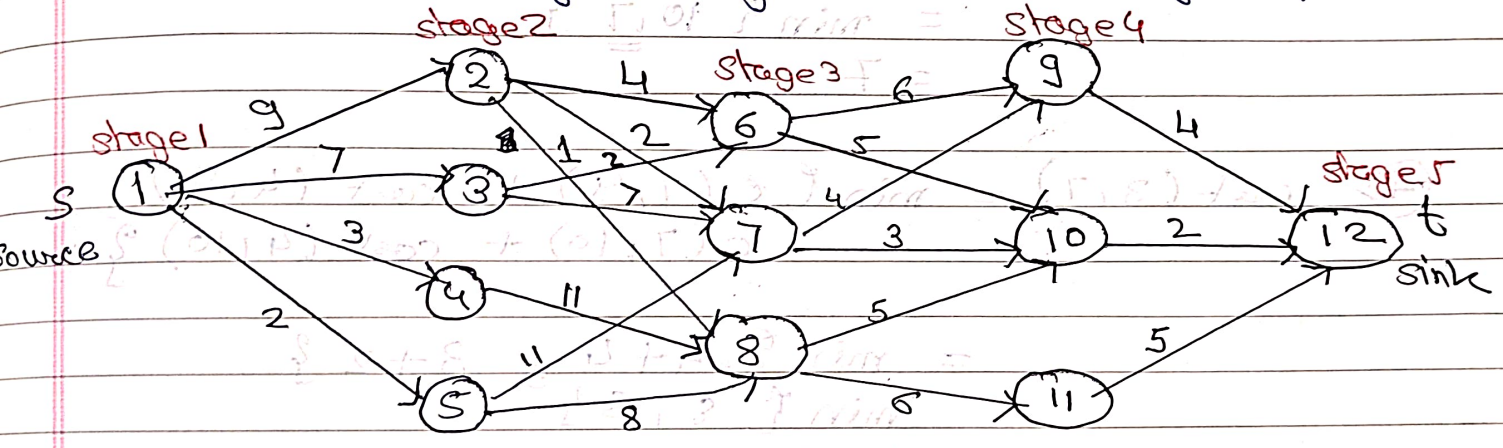
Backward Approach

① forward approach

$$cost(i,j) = \min\{c(j,l) + cost(i+1, l)\}$$

i → stages, j → vertex No.

Note:- Dynamic programming is used to get optimization



V	1	2	3	4	5	6	7	8	9	10	11	12
cost	16	7	9	18	15	7	5	7	4	2	5	0
d	2/3	7	6	8	8	10	10	10	12	12	12	12

2 or 3

Note:- In forward approach, we start from back vertex (backward) as last stage.

(1) cost (stage no, vertex No) = direct cost destination

(2) cost (5, 12) = 0 — stage 5

(3) cost (4, 9) = 4
cost (4, 10) = 2
cost (4, 11) = 5
} stage 4

(4) cost (3, 6) = ~~there~~
There is two path, so to calculated min cost we do the following thing.
cost (3, 6) = min { c (6, 9) + cost (4, 9),
c (6, 10) + cost (4, 10) }

= min { 6+4, 5+2 }
= min { 10, 7 }
= 7

(5) cost (3, 7) = min { c (7, 9) + cost (4, 9)
c (7, 10) + cost (4, 10) }
graph ↓ table ↓

= min { 4+4, 3+2 }
= min { 8, 5 }
= 5

(6) cost (3, 8) = min { c (8, 10) + cost ( )
c (8, 11) + cost ( ) }

= min {

Stage 2

(4) $cost(2,2) = \min\{ c(2,6) + cost(3,6),$
$c(2,7) + cost(3,7),$
$c(2,8) + cost(3,8) \}$

$= \min\{4+7, 2+5, 1+7\}$
$= \min\{11, 7, 8\}$
$= 7$

$cost(2,3) = \min\{ c(2,6) + cost(4,6),$
$c(2,7) + cost(4,7) \}$
$= \min\{7+7, 5+5\}$
$= \min$
$= 9$

doubt

~~$cost(2,4) = \min\{ c($~~ $cost$

$cost(2,3) = \min\{ c(3,6) + cost(4,6), c(3,7) + cost(4,7) \}$

$=$
$= 9$

$cost(2,4) = \min\{ c(4,8) + cost(3,8) \}$
$= \min\{11, 7\}$
$= 18$

$cost(2,5) = \min\{ c($

Multistage graph using Backward Approach (Multistage graph)

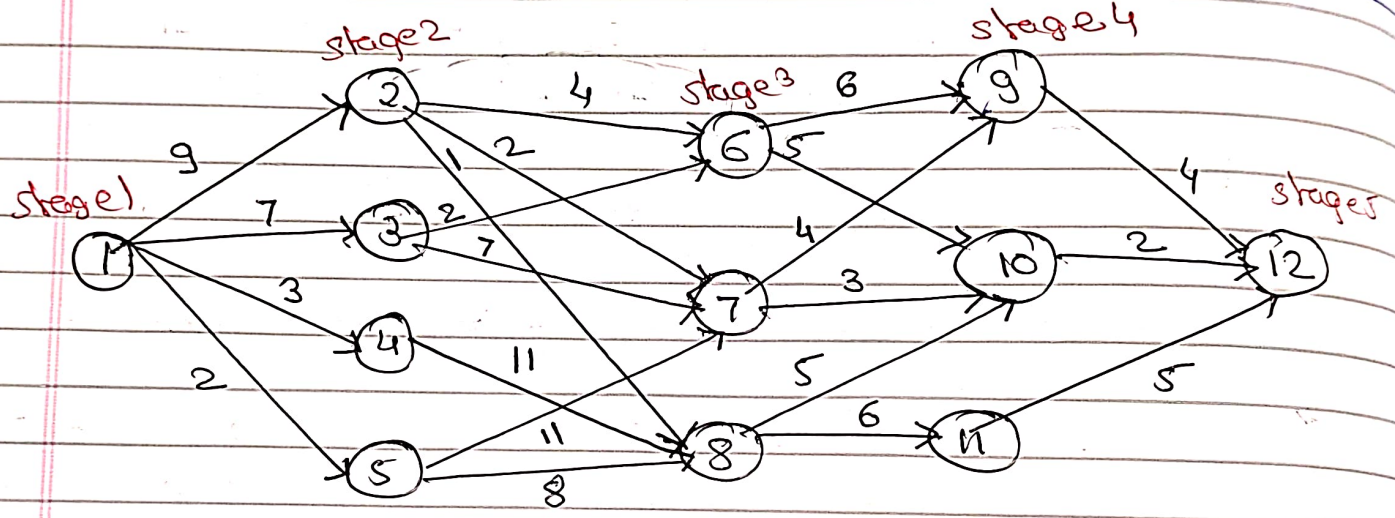
Formula

~~cost(i,j)~~

$$bcost(i,j) = \min\{ bcost(i-1, l) + c(l,j) \}$$

stage — vertex

Q1.



Soln:-

V	1	2	3	4	5	6	7	8	9	10	11	12
cost	0	9	7	3	2	9	11	10	15	14	16	16
d	1	1	1	1	1	3	3	2/5/6/7	6/7	8	10	

stage1 bcost(1,1) = 0

stage2 bcost(2,2) = 9
bcost(2,3) = 7
bcost(2,4) = 3
bcost(2,5) = 2

Note: Cost coard go to table
bcost value see from table

6 pa kaha se aya 2 & 3

stage 3:- $bcost(3,6) = \min\{ bcost(2,2) + c(2,6),\ bcost(2,3) + c(3,6) \}$ (stage, vertex)

$= \min\{9+4, 7+2\}$

$= \min\{13, 9\} = 9$

$bcost(3,7) = \min\{ bcost(2,2) + c(2,7),\ bcost(2,3) + c(3,7),\ bcost(2,5) + c(5,7) \}$

$= \min\{9+2, 7+7, 2+11\}$

$= \min\{11, 14, 13\}$

$= 11$

$bcost(3,8) = \min\{ bcost(2,2) + c(2,8),\ bcost(2,4) + c(4,8),\ bcost(2,5) + c(5,8) \}$

$= \min\{9+1, 3+11, 8+8\}$

$= \min\{10, 14, 16\}$

$= \{10, 10\}$

stage 4:- $bcost(4,9) = \min\{ bcost(3,6) + c(6,9),\ bcost(3,7) + c(7,9) \}$

$= \min\{9+6, 11+4\}$

$= \min\{15, 15\}$

$bcost(4,10) = \min\{ bcost(3,6) + c(6,10),\ bcost(3,7) + c(7,10),\ bcost(3,8) + c(8,10) \}$

$= \min\{9+5, 11+3, 10+5\}$

$= \min\{14, 14, 15\}$

$= 14$

Note:- In forward approach, we start from end of the table.
In backward approach, we start from starting of table.

$bcost(4,11) = \min\{ bcost(3,8) + c(8,11) \}$
$= \min\{10 + 6\}$
$= \min\{16\}$
$= 16$

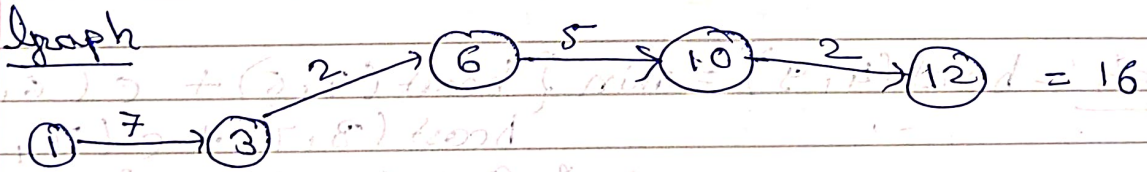
$bcost(5,12) = \min\{ bcost(4,9) + c(9,12),$
$bcost(4,10) + c(10,12),$
$bcost(4,11) + c(11,12) \}$

$= \min\{15+4, 14+2, 16+11\}$
$= \min\{19, 16, 27\}$
$= \min\{16\}$
$= 16$

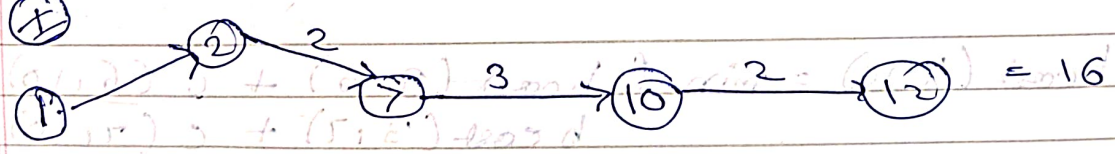
Min.m cost from source to dist.n 1-12

12 - 10 <
- 6 - 3 - 1 (path1)
- 7 - 2 - 1 (path2)

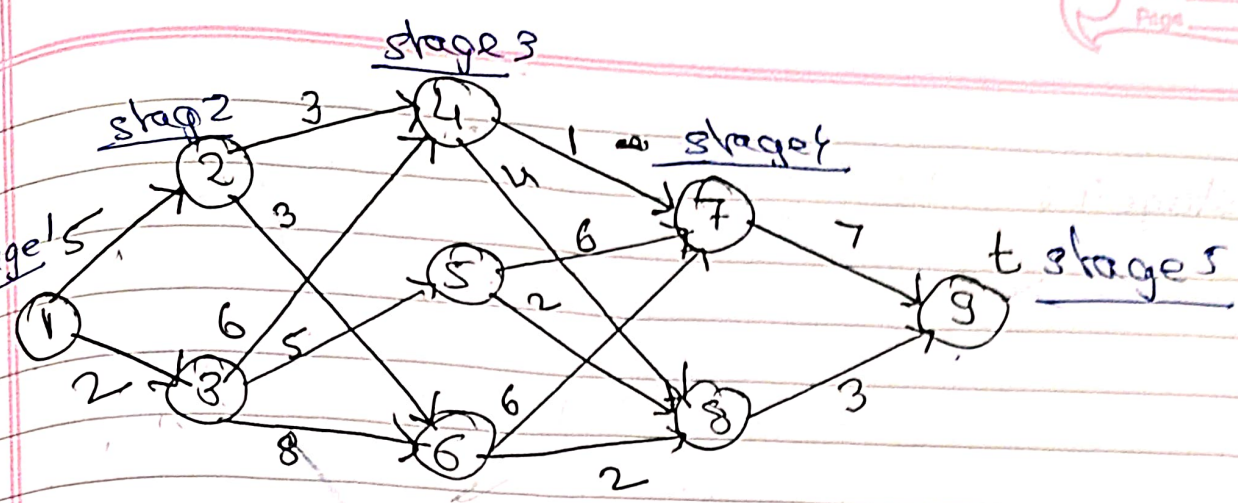
Graph



OR

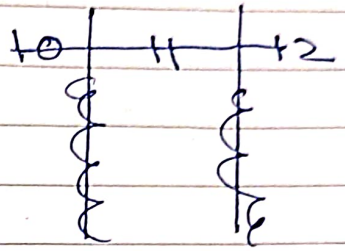


Q2.



Soln:-

V	1	2	3	4	5	6	7	8	9
cost	0	5	2	8	5	8	9	7	10
d	1	1	1	2/3	3	2/3	4	5	8



stage 1:- bcost(1,1) = 0

stage 2:- bcost(2,2) = 5
bcost(2,3) = 2

stage 3:- bcost(3,4) = min{ bcost(2,2) + c(2,4)
bcost(2,3) + c(3,4) }
= min{ 5 + 3, 2 + 6 }
= min{ 8, 8 }
=

bcost(3,5) = 5
min{ 2 + 3 }

bcost(3,6) = min{ bcost(2,2) + c(2,6)
bcost(2,3) + c(3,6)
= { 5 + 3, 2 + 8 }
= { 8, 10 }
= 8

Stage 4 :- bcost(4,7) = min{ bcost(3,6) + c(6,7)
bcost(3,5) + c(5,7),
bcost(3,6) + c(6,7) }

= min{ 8+1, 5+6, 8+6 }
= min{ 9, 11, 14 }
= min{ 9 }

bcost(4,8) = min{ bcost(3,4) + c(4,8),
bcost(3,5) + c(5,8)
bcost(3,6) + c(6,8) }

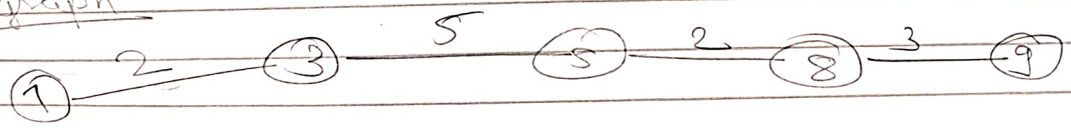
= min{ 8+4, 5+2, 8+2 }
= min{ 12, 7, 10 }
= 7

Stage 9 :- bcost = min{ bcost(4,7) + c(7,9)
bcost(4,8) + c(8,9)
= min{ 9+7, 7+3 }
= min{ 16, 10 }
= 10

Min.m cost from source to dist.n 1-9

1-9 ⇒ 8-5-3-1

Graph



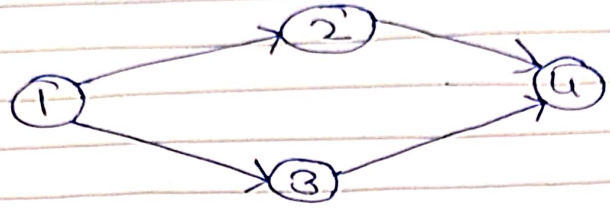
Dynamic divide the program / problem into to sub-problem and issue to solve the another problem is called dynamic

Dynamic Programming

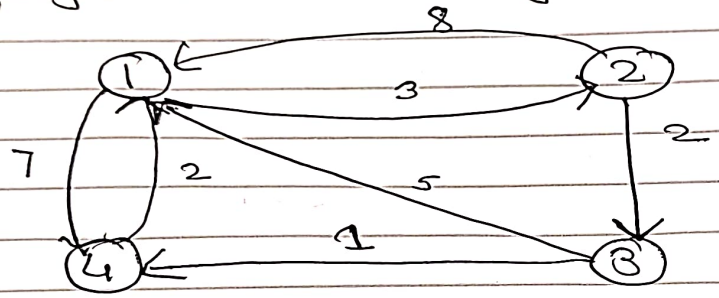
All pair shortest path (Floyd-Warshall Algorithm)

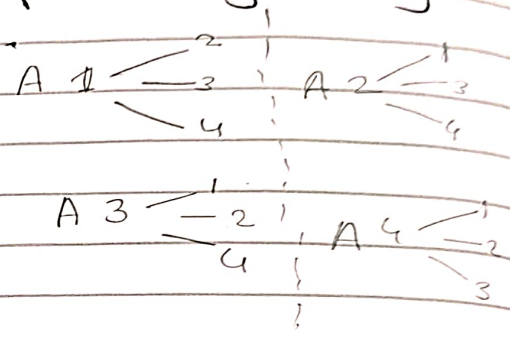
single All pair

Single source shortest path



Q Calculate All pair shortest path by using floyd Warshall Algorithm.





basic matrix

$$A^0 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{bmatrix} \end{array}$$

- Diagonal matrix is always 0 i.e 1-1, 2-
- if direction is not the write '∞'

$$A^k[i,j] = \min\{ A^{k-1}[i,j] , A^{k-1}[j,k] + A^{k-1}[k$$

Floyd program

classmate
Date
Page

$A^1 =$

	1	2	3	4
1	0	3	∞	7
2	8	0	2	15
3	8	8	0	1
4	2	5	∞	0

Note: 1st row & 1st column (same as A^1) remain as it is & diagonal value is zero

(1-1=0)

$A^1[2,3] = \min\{A^0[2,3], A^0[2,1] + A^0[1,3]\}$
$= \min\{2, 8+\infty\}$
$= 2$

$A^1[2,4] = \min\{A^0[2,4], A^0[2,1] + A^0[1,4]\}$
$= \min\{\infty, 8+7\}$
$= 15$

$A^1[3,2] = \min\{A^0[3,2], A^0[3,1] + A^0[1,2]\}$
$= \min\{$
$= 8$

$A^1[3,4] = \min\{A^0[3,4], A^0[3,1] + A^0[1,4]\}$
$= \min\{1, 5+7\}$
$= 1$

$A^1[4,2] = \min\{A^0[4,2], A^0[4,1] + A^0[1,2]\}$
$= \min\{$
$= 5$

$A^1[4,3] = \min\{A^0[4,3], A^0[4,1] + A^0[1,3]$
$= \infty$

Matrix A^2 =

i \ j	1	2	3	4
1	0	3	5	7
2	8	0	2	15
3	5	8	0	1
4	2	5	7	0

Note 2nd process & 2nd column as it is

$A^2[1,3] = \min\{A^1(1,3), A^1(1,2) + A^1(2,3)\}$

$= \min\{\infty, 3+2\}$

$= 5$

$A^2[1,4] = \min\{A^1(1,4), A^1(1,1) + A^1(1,4)\}$

$= \min\{\infty, 0 +$

$=$

$A^2[3,1] = \min\{A^1(3,1), A^1(3,1) + A^1(1,$

Q. How to time complexity is $O(n^3)$ of floyd warshall algo.

```
for(k=1; k<=n; k++)
{
    for(i=1; i<=n; i++)
    {
        for(j=1; j<=n; j++)
        {
            A^k[i,j] = min{A^(k-1)(i,j), A^(k-1)(i,k) + A^(k-1)(k,j)}
        }
    }
}
```

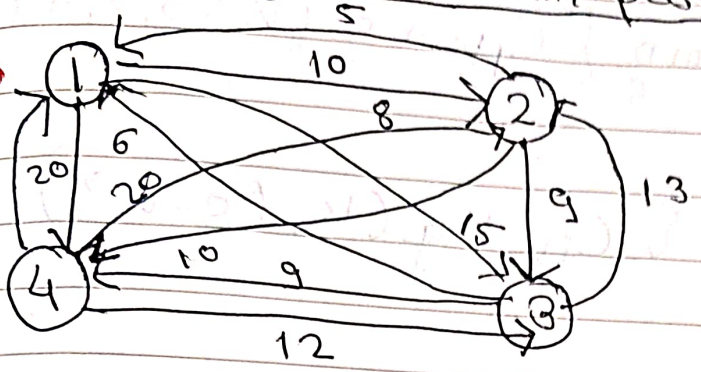
Time complexity is $O(n^3)$

There are three loop i, j, k

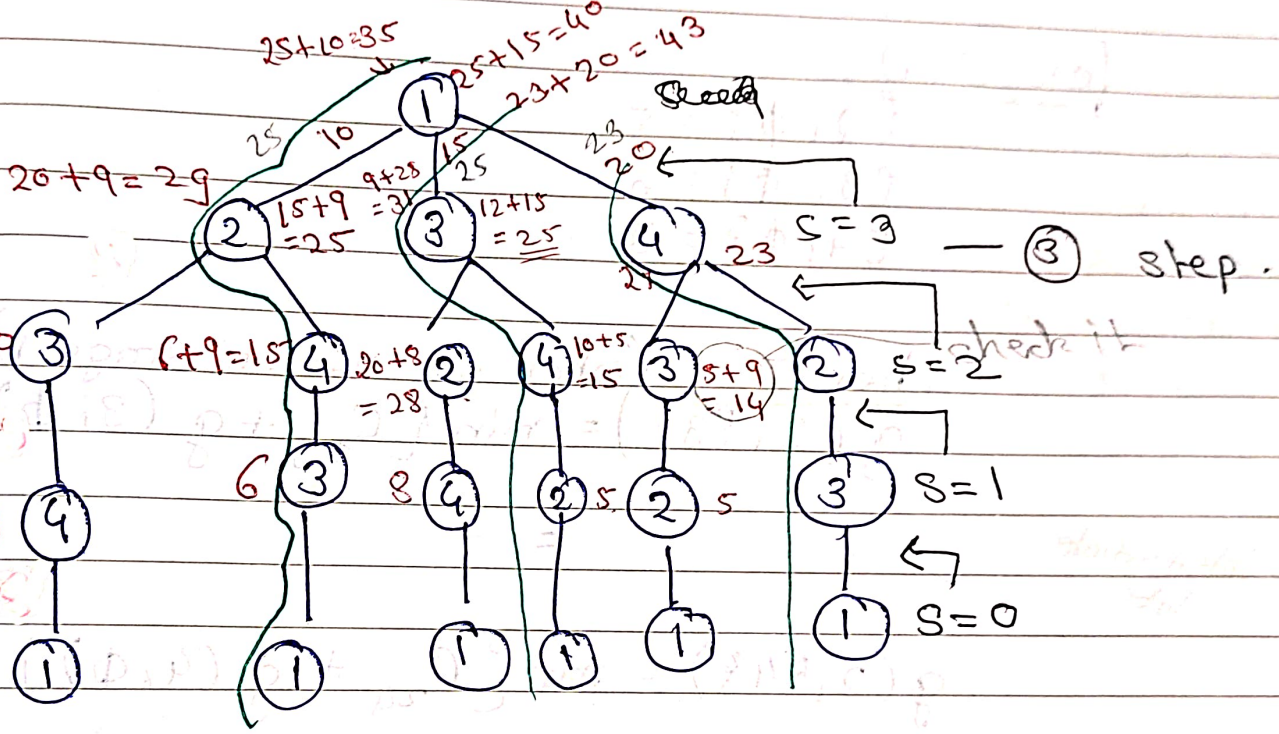
$$A^3 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & & 5 & \\ & 0 & 2 & \\ & & 0 & \\ & & 7 & 0 \end{bmatrix} \end{array} = \begin{bmatrix} 0 & 3 & 5 & 6 \\ 7 & 0 & 2 & 3 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix}$$

Dynamic Programming

Travelling salesman problem (TSP)



— ① step



— ③ step.

check it

	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

write the matrix from graph — ② step

path 1 - 2 - 4 - 3 - 1 = 35 (min.m cost)

formula (M-2)

$$g(1,\{2,3,4\}) = \min_{k \in S} \{ C_{1k} + g(k,\{2,3,4\} - \{k\}) \}$$

$$g(i,\{S\}) = \min_{k \in S} \{ C_{ik} + g(k,\{S-k\}) \}$$

for $S = 0$,

$g(2,\{\phi\}) = 5$

$g(3,\{\phi\}) = 6$

$g(4,\{\phi\}) = 8$

we consider ϕ, because there is no node between 2 to 1, 3 to 1, 4 to 1 ($\phi = 1$)

when $S = 1$ (when we are on stage 1)

$$g(2,\{3\}) = \min \{ C_{23} + g(3,\phi) \}$$

$$=$$

$$g(2,\{4\}) = \min \{ C_{24} + g(4,\phi) \}$$

$$=$$

$$g(3,\{2\}) = \min \{ C_{32} + g(2,\phi) \}$$

$$=$$

$$g(3,\{4\}) = \min \{ C_{34} + g(4,\phi) \}$$

$$=$$

k = intermediate state

Write formula for spanning tree
Write formula for Dynamic programming (travelling salesman problem)

$g(4,\{2\}) = \min\{C_{42} + g(2,\phi)\}$

=

$g(4,\{3\}) = \min\{C_{43} + g(3,\phi)\}$

=

when $s=2$

$g(2,\{3,4\}) = \min\{C_{23} + g(3,\{4\}), C_{24} + g(4,\{3\})\}$

=

$g(3,\{2,4\}) = \min\{C_{32} + g(\{4\},\{3\}), C_{34} + g(4,\{2\})\}$

=

=

$g(4,\{2,3\}) = \min\{C_{42} + g(4,\{2\}), C_{43} + g(4,\{3\})\}$

=

=

when $s=3$

$g(1,\{2,3,4\}) = \min\{C_{12} + g(2,\{3,4\}), C_{13} + g(3,\{2,4\}); C_{14} + g(4,\{2,3\})\}$

(value see from graph; value see from above answer)

$= \min\{35, 40, 43\}$

$= 35$

Final path:- 1 - 2 - 4 - 3 - 1

Dynamic Programming

0/1 Knapsack problem

m = capacity of bag = 8
n = no. of object = 4
P = profit of object = {1, 2, 5, 6}
w = wt of an object = {2, 3, 4, 5}

write all 1st row & column 0.

we have 4 object

Pi	Wi
1	2
2	3
5	4
6	5

n \ m	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1	1
2	0	0	1	2	2	3	3	3	3
3	0	0	1	2	5	5	6	7	7
4	0	0	1	2	5	6	6	7	8

formula:- $V[i, w] = \max\{V[i-1, w], V[i-1, w-w[i]] + P[i]\}$

for object 1, Here wt is 2 & capacity is 1 ∴ m=1 & n=1
that's why it is 0

Here 3 object bcz 1+2 = 3

{x1, x2, x3, x4}
{w1, w2, w3, w4}
{0|1, 0|1, 0|1, 0|1}

More profit = 8

{ x1, x2, x3, x4 }
{ w1, w2, w3, w4 }
{ 0, 1, 0, 1 } here consider object 2 & 4 that having wt 3 & 5 respectively, by adding these 2 & 4, we get 8 as a profit.

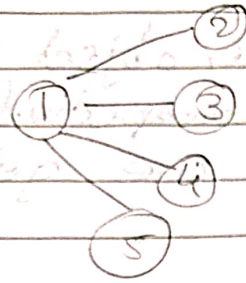
Backtracking

- ~~Breadth first search Algorithm~~

[M-2] Theory question: ① what is dynamic
② Diff. bet dynamic & greedy
③ can we apply single source to all pair

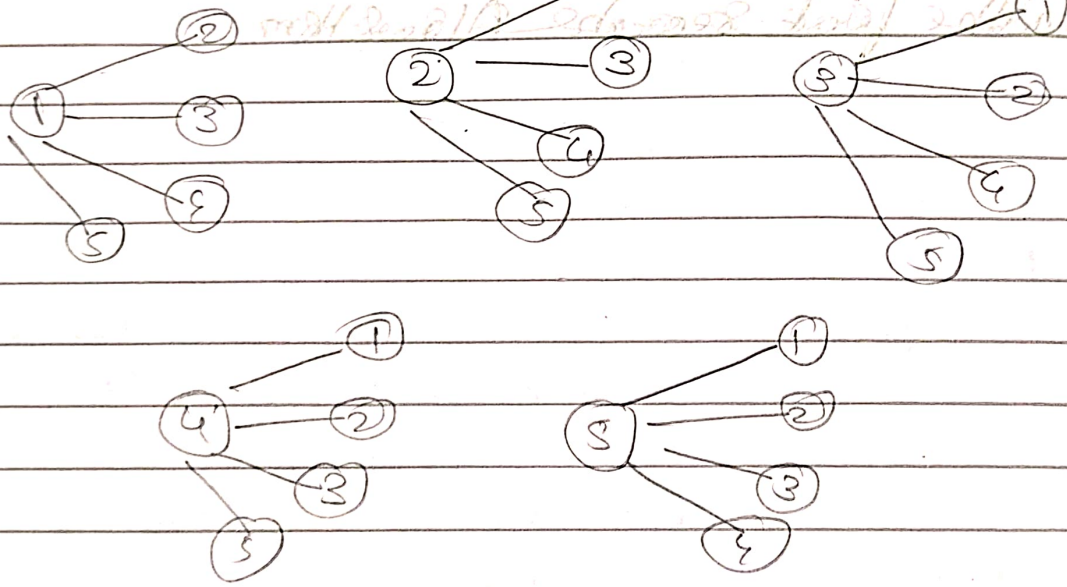
→ ans:-

- Single source



I want min cost from single cost

- All pair source



Yes, we can apply single source 5 times to get all pair.

Yes, we cal.n single source to cal.n all pair, but we have to calculated 5 times
It required n time complexity
It will increase time complexity.

L-5 :-

Backtracking

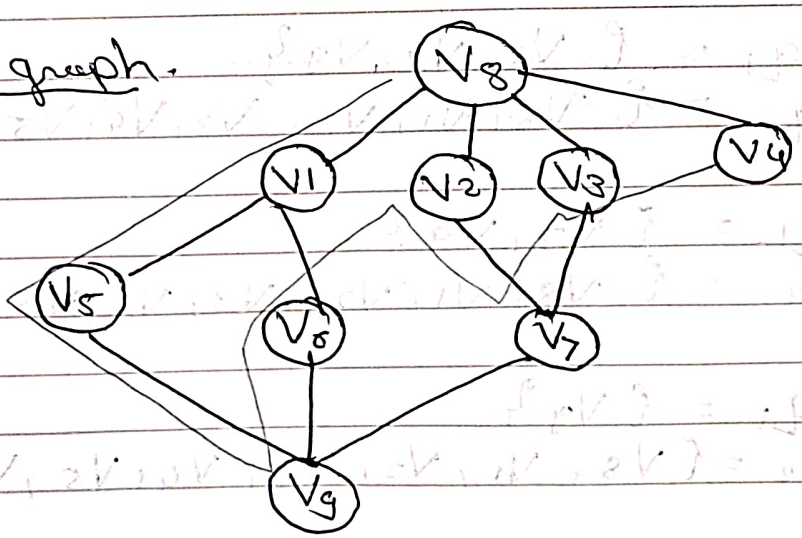
- Breadth first search Algorithm
- Depth first search Algorithm.

BFS algorithm

Rules -
It travel the graph breath-wise, it used queue to remain to get the next vertex to start a search.

DFS
Rules / algorithm
step 1 :- Visit adjacent unvisited vertex, marke it as visited, display it insert in the queue
step 2 :- If no adjacent vertex is found remove first vertex from the queue
step 3 :- Repeat rule 1 to 2, until the queue is empty.

Eg 1] Consider graph.



- Let start examine the graph V_8
- Create two queues q_1 & q_2
  the q_1 hold all the nodes that are process while q_2 hold all the nodes that are deleted from q_1

① $q_1 = \{V_8\}$
$q_2 = \{\ \}$

② $q_1 = \{V_1, V_2, V_3, V_4\}$ add child of V_8 to q_1 & delete V_8 from q_1 & insert to q_2
$q_2 = \{V_8\}$

Note: Add Element at end of queue.

③ $q_1 = \{V_2, V_3\ V_4\ V_5\ V_6\}$
$q = \{V_8, V_1\}$

④ $q_1 = \{V_3, V_4, V_5, V_6, V_7\}$
$q_2 = \{V_8, V_1, V_2\}$

⑤ $q_1 = \{V_4, V_5, V_6, V_7\}$ Here V_7 is child of V_3 also, but V_7 is ready in queue so, don't add
$q_2 = \{V_8, V_1, V_2, V_3\}$

⑤ $q_1 = \{V_5, V_6, V_7\}$
$q_2 = \{V_8, V_1, V_2, V_3, V_4\}$

⑥ $q_1 = \{V_6, V_7, V_9\}$
$q_2 = \{V_8, V_1, V_2, V_3, V_4, V_5\}$

⑦ $q_1 = \{V_7, V_9\}$
$q_2 = \{V_8, V_1, V_2, V_3, V_4, V_5, V_6\}$

⑧ $q_1 = \{V_9\}$
$q_2 = \{V_8, V_1, V_2, V_3, V_4, V_5, V_6, V_7\}$

(9) $q_1 = \{\}$

$q_2 = \{V_8, V_1, V_2, V_3, V_4, V_5, V_6, V_7, V_9\}$

DFS

step1:- Visit the adjustance / adjacent

step2:- We no adjacent vertex is found, pop up the vertex from a stack.

step3 Repeat 1 & 2, until stack is empty

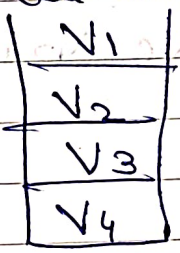
Eg 2] ⟹ ① start from node V_8 & push into stack.

diagram see of Eg. 1

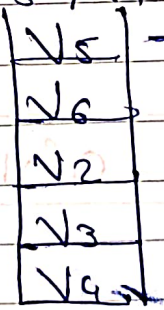
② pop V_8 & print

print V_8

& add all adjustion node of V_8 in stack.

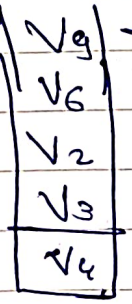


③ print: V_8, V_1



④ print: V_8, V_1, V_5

pop V_5 & add adjacent element of V_5 in stack i.e V_9



Note:- Diff. betn dfs & bfs.

backtracking ↓ queue

↳ stack

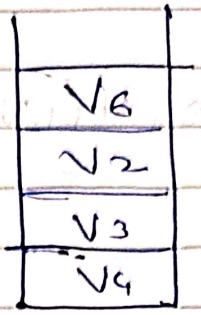
Note

DFS :-

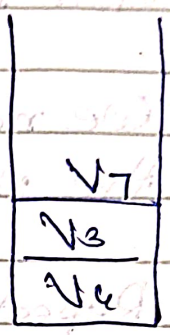
All nodes connected to is used

⑤ print V8, V5, V9

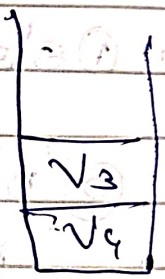
pop V9 & insert adjacent element, but is ready add so need to add that



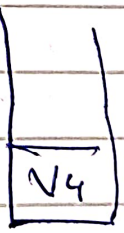
⑥ print V8, V5, V9, V6, V2



⑦ print V8, V5, V9, V6, V2, V7



⑧ print V8, V5, V9, V6, V2, V7, V3



⑨ print V8, V5, V9, V6, V2, V7, V3

Time complexity :- O(V+E)

Backtracking

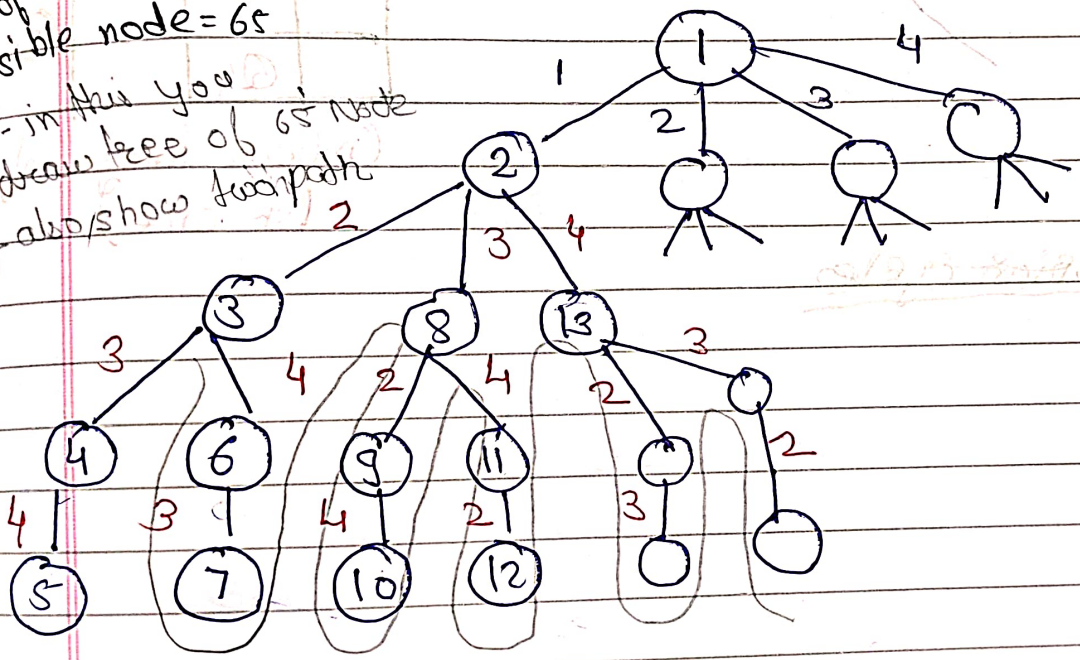
1. Basic algorithm — BFS, DFS
2. N-queen problem — 4 queen, 8 queen
3. N-colaring problem
4. Hamiltonian cycle

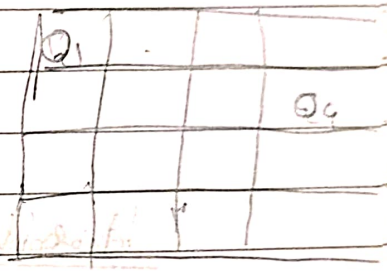
In 4 queen, 4-Q {Q_1, Q_2, Q_3, Q_4} → Explicit constraint
No two queen attack in row, column, & diagonal.
↳ implicit constraint

Note:- State Space tree (There is possibility $^{16}C_4$)

No. of possible Node = 65

Note:- In this you draw tree of 65 Node & also show two path



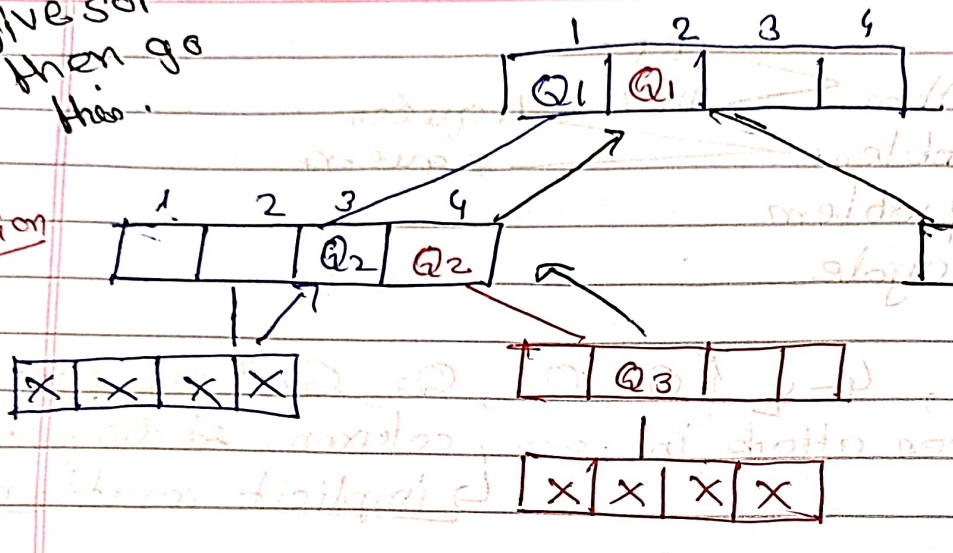


Note:- $1+4+4\times3+4\times3\times2+4\times3\times2\times1 = 65$

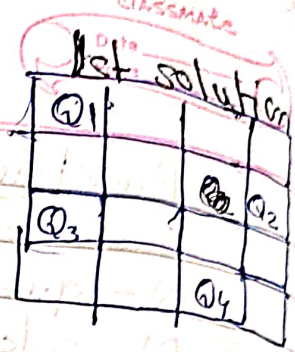
formula for 4 queen

$$65 = 1 + \sum_{i=0}^{3} \left[\prod_{j=0}^{i} (4-j) \right]$$

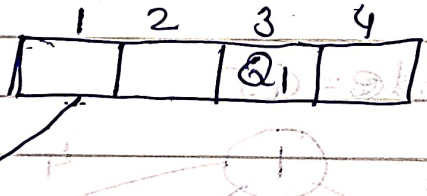
Note:-
Only give soln
~~if~~ then go
this.

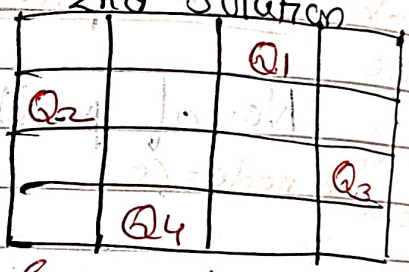


1 solution



2nd solution

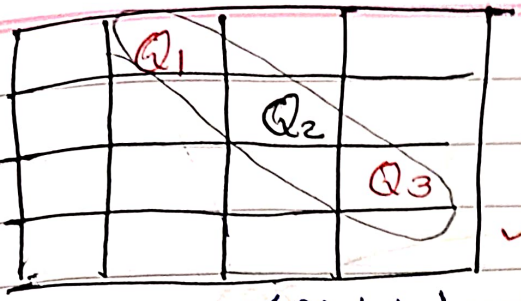




(Mirror image of 1st one)

~~Hamiltonian Cycle~~

Note:-



(Right to bottom)

× $Q_2 = (2,3) = -1$ (Left to Bottom)
$Q_1 = (1,2)$ or $Q_3 = (3,4) = -1$

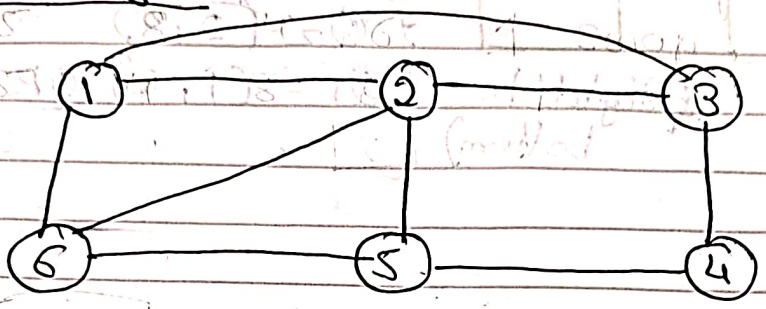
✓ $Q_2 = (2,3)$
$Q_1 = (1,4)$ OR $Q_3 = (3,2), (4,1)$

Q. What is diff. betn TSP & Hamiltonian cycle.

Q Identify this problem is come under which category i.e NP class, P class, NP comple, NP hard.

Backtracking

Hamiltonian cycle



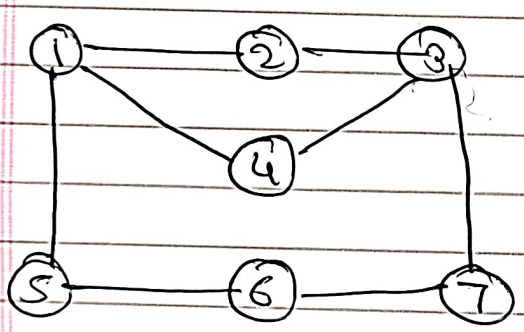
1 - 2 - 3 - 4 - 5 - 6 - 1

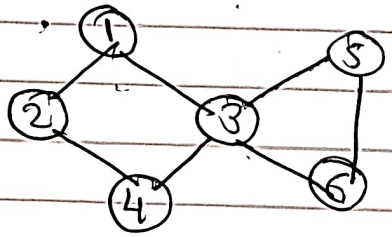
1 - 6 - 5 - 4 - 3 - 2 - 1

not accept bcz it is same

1 - 3 - 4 - 5 - 2 - 6 - 1

2 - 3 - 4 - 5 - 6 - 1 - 2

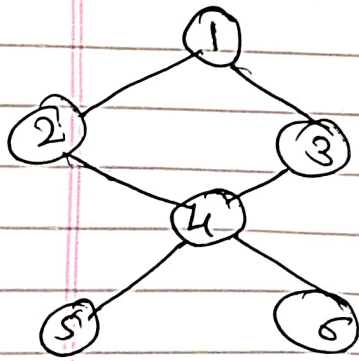




Path 1-2-3-7-6-5
but here pt 4 is missing

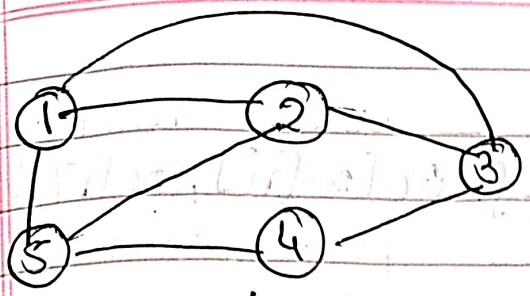
Path 1-2-4-3-5-6
Here 1 is missing

Articulation point



Pendent point
(it does not go forward)

Path 1-2-3-4-5
1-2-3-4-6



G (In this problem, we fixed the starting vertex)

G =	1	2	3	4	5
1	0	1	1	0	1
2	1	0	1	0	1
3	1	1	0	1	0
4	0	0	1	0	1
5	1	1	0	1	0

Algorithm Hamiltonian (k)
{
  do
  { NextVertex (k);
    if x[k] == 0
      return;
    if (k == n)
      print (x[1:n])
    else
      Hamiltonian (k+1)
  } while true;
}

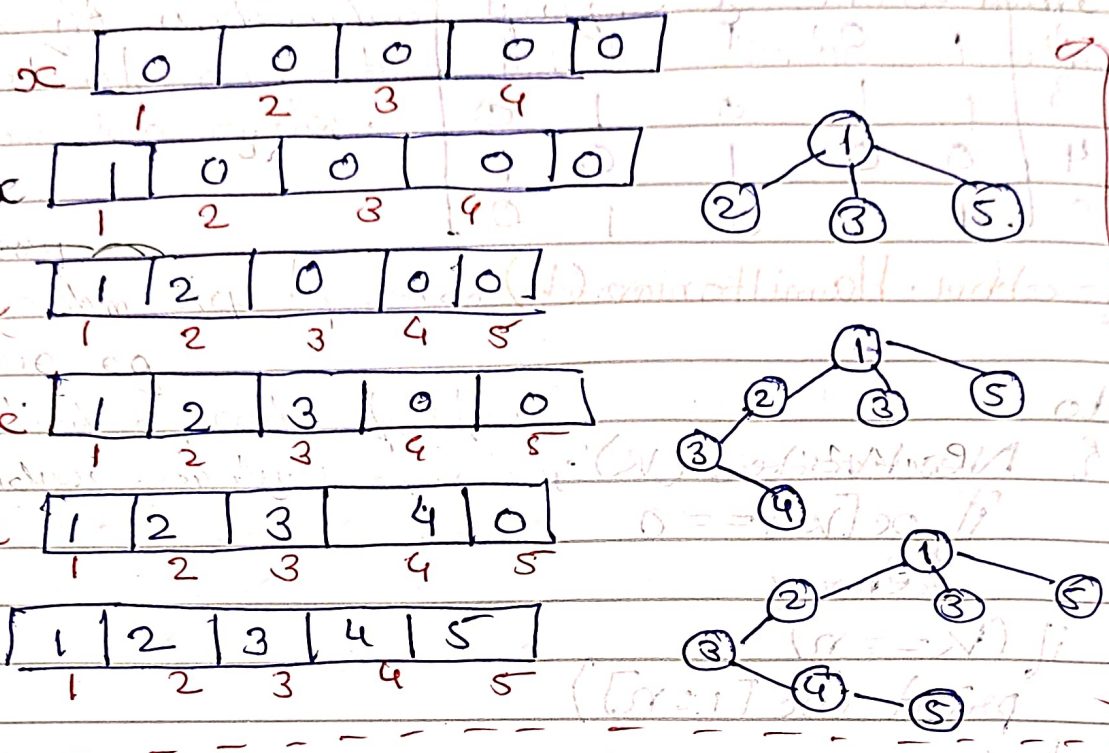
Algorithm NextVertex (k)
{
  do
  { x[k] = (x[k] + 1) mod (n+1)
    if (x[k] == 0) return
    if (G[x[k-1], x[k]] ≠ 0
    { for j = 1 to k-1 do
      if (x[j] == x[k])
        break;

How backtracking used to find Hamiltonian alg.

Q. General state-space tree for three sol.n ?

Q How Hamiltonian work with back tracking ?

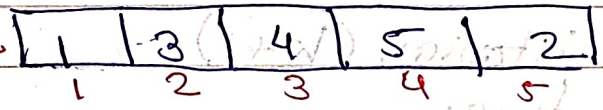
```
if (j == k)
if (k < n or (k == n) && G[x[n], x[1]] != 0
    return;
while (true);
```

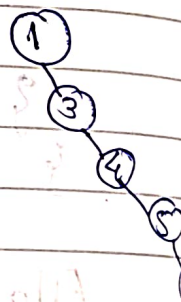


1st path = 1st solution

here check 1st in previous there is 2 & 2nd also check there is path bet^n 1 & 2

In last step, u have to check 3 cond.^n 1st compare with each & every element we 5 is present or not after that check there is path or not, after that 3rd cond.^n check is there is path bet^n 5 to 1

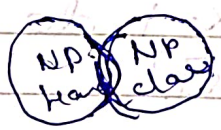


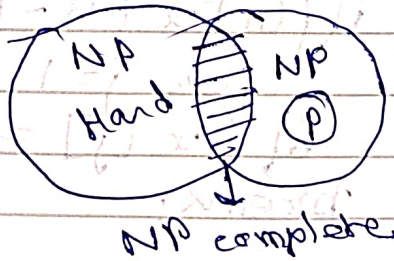


~~#~~ P, NP, NP Hard, NP-complete

P: polynomial time taken algorithm $O(1) \rightarrow O(n^k)$

NP: Nonpoly





P ⊆ NP

- Exponential type algorithm.
- Polynomial type algorithm.

Page No:
Date: 31 / 10 / 25

* NP Hard - NP Complete

· Deterministic Polynomial time taking algorithm:

Linear Search - n
Binary Search - $\log n$
Merge Sort - $n \log n$
Insertion Sort - n^2

0/1 knapsack - 2^n
TSP - 2^n
N-queen problem - 2^n
Graph Coloring - 2^n
Hamiltonian - 2^n
} NP hard

task is to reduce ~~polynomial time~~ exponential time into polynomial time and polynomial time into single

· Non-deterministic Polynomial time taking algo

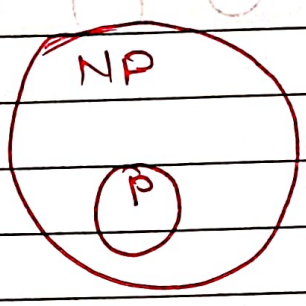
```
algo. N search (A, n, key)
{
    j = choice ();        → 1 unit of time
    if (key = A[j])
    {
        write (j);
        success ();       → 1 unit of time
    }
    write (0)
    failure ();           → 1 unit
}
```

in this we have made an algo. in which the result is found but don't know how, hence, it is a Non-deterministic polynomial algo.

Page No:
Date: 31 / 10 / 25

NP Hard

Time taking to solve the problem (exponential time) or the problem does not get solved.



Soln → Satisfiability Problem (Common Problem) Similar to All NP hard

Belief before

0/1 knapsack 2^n
TSP 2^n
Nqueen 2^n

We are defining one problem — i.e, CNF Propositional calculus formulae using boolean

$x_i = (x_1, x_2, x_3)$

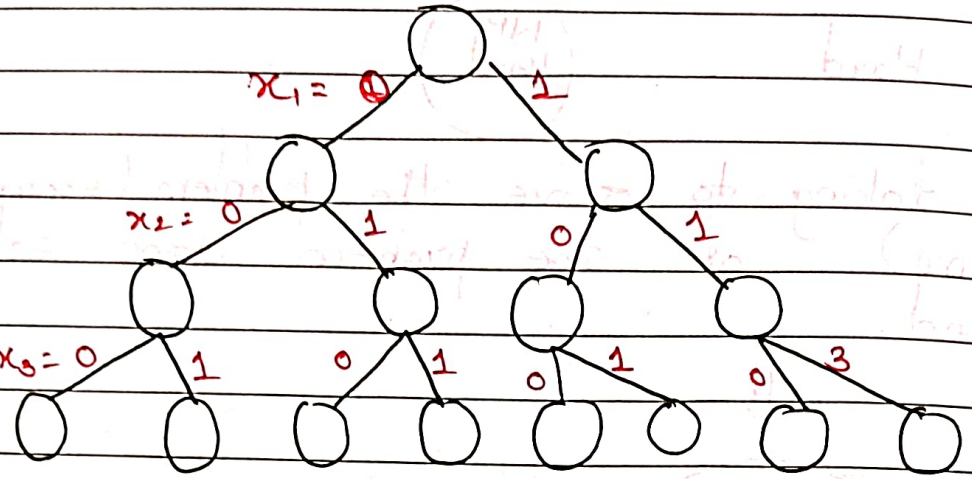
$T \Rightarrow (x_1 \vee \bar{x_2} \vee x_3) \wedge (\bar{x_1} \vee x_2 \vee \bar{x_3})$

we will require 2^3 boolean values $\Rightarrow 2^n$ time
hence it can be called as NP hard.

Q. Show the relation between 0/1 knapsakle & Satisfiability problem.

Page No:
Date: 31/10/25

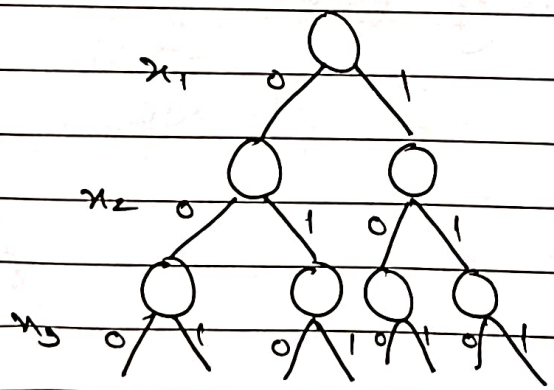
Q. ~~Show~~ ~~it~~ What is satisfiability prob.?



0/1 knapsack

(x_1, x_2, x_3)

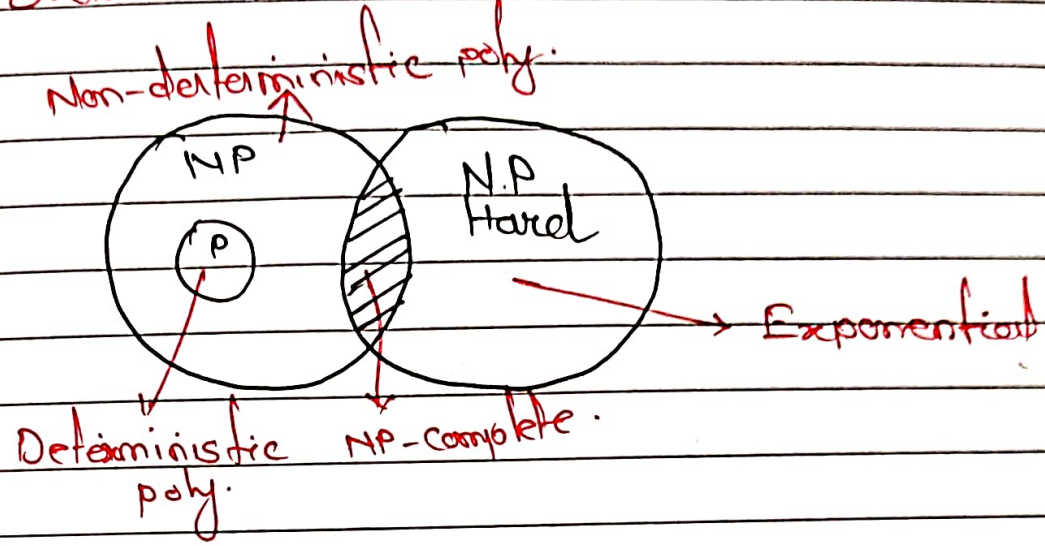
$(0|1, 0|1, 0|1)$



here we have shown the relation between the satisfiability & 0/1 knapsack

Page No:
Date: 31 / 10 / 25

Q. Show P ⊆ NP



NP hard $\xrightarrow{\text{expo}^{nt}}$ Non-deterministic poly. $\Rightarrow$ NP Complete.

Deterministic poly. time. Non-deterministic poly. time. Expo. time.